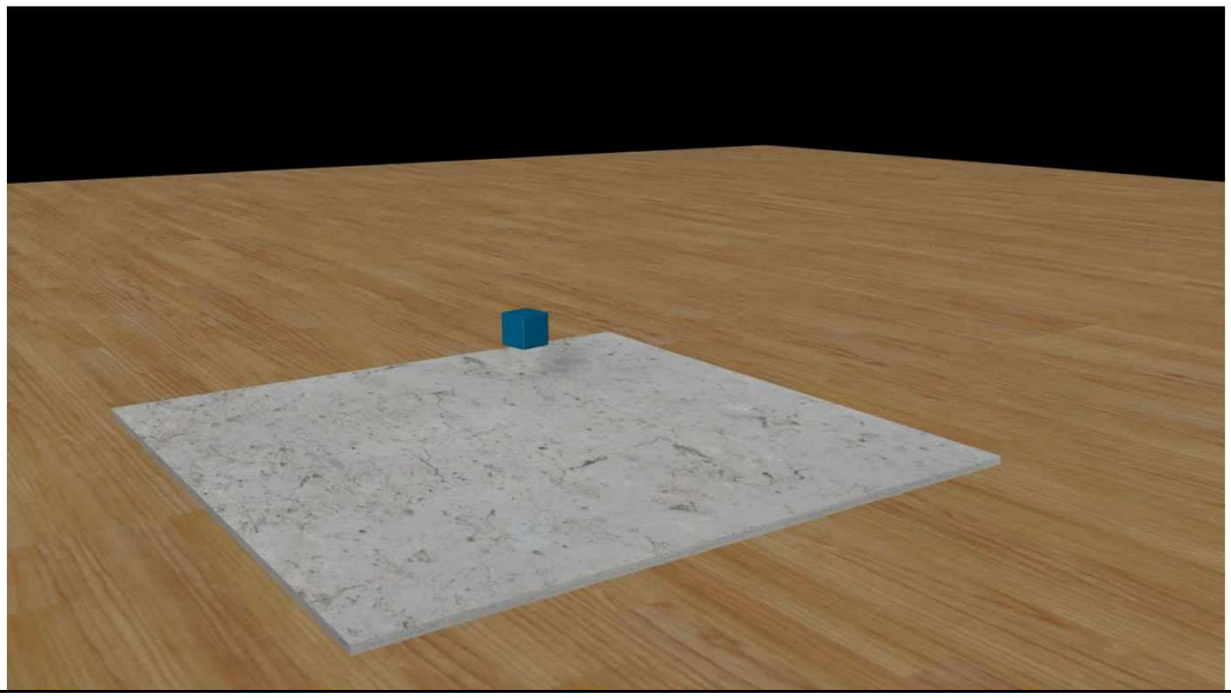
Rigid-body Dynamics and Contact Mechanics

David Hahn

1

Computer Animation and Simulation course, TU Wien, spring 2026

Motivation



2

First some motivational examples of simulations involving contacts

This is a rigid body simulation using the Bullet-Physics library

We won’t talk about the fracture part here

Motivation

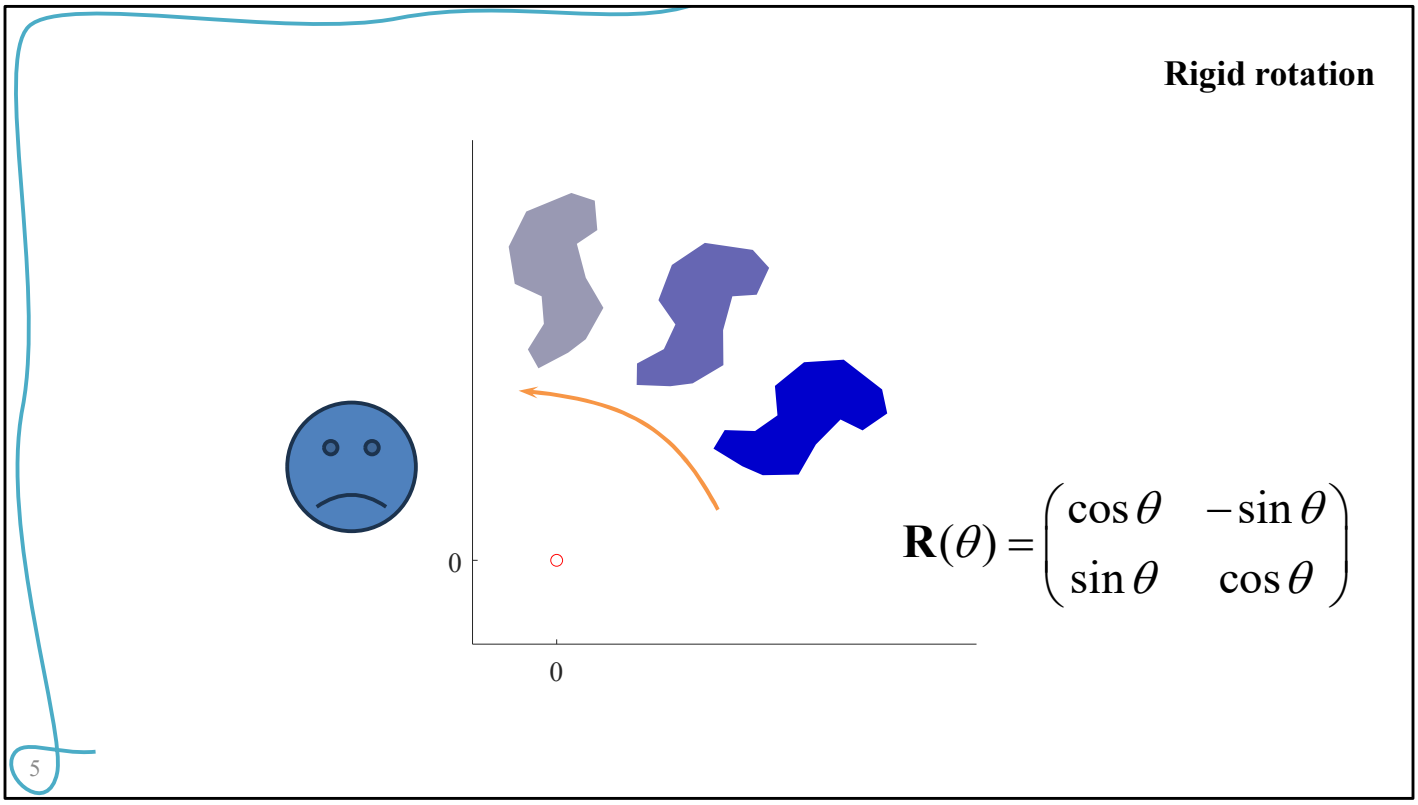
3

C. Giles et al., Augmented Vertex Block Descent, ACM TOG 44, 2025

Here's a result from a more recent paper that runs in (near) real time – we'll return to this paper at the end

Outline

- Rigid body motion
- Contact constraints
- Coulomb friction
- Hard vs. soft constraints

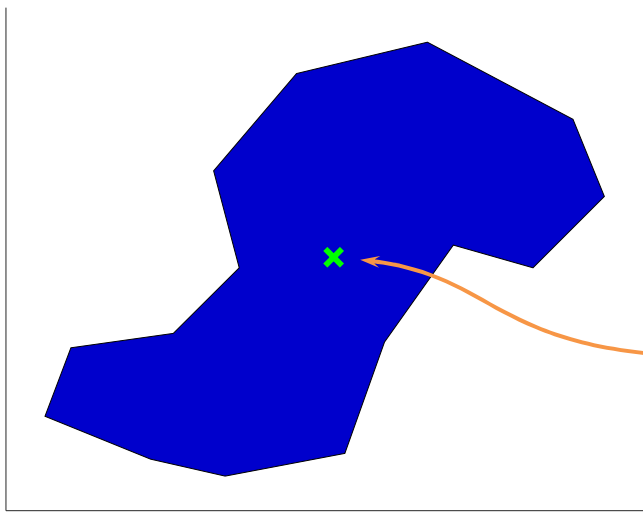


Let's first review some basics about how rigid motion can be formalized.

(Rigid of course means, that things move around without deforming, i.e., they remain the same in a body-local reference coordinate frame.)

Simply rotating some object (like this polygon here) by applying a standard rotation matrix to all the point coordinates will rotate it around the origin … unfortunately, that's not really what we want for describing physical motions …

Rigid rotation – centre of mass



Mass:

$$m = \int_{\Omega_0} \rho(\mathbf{p})\, d\mathbf{p}$$

Centre of mass:

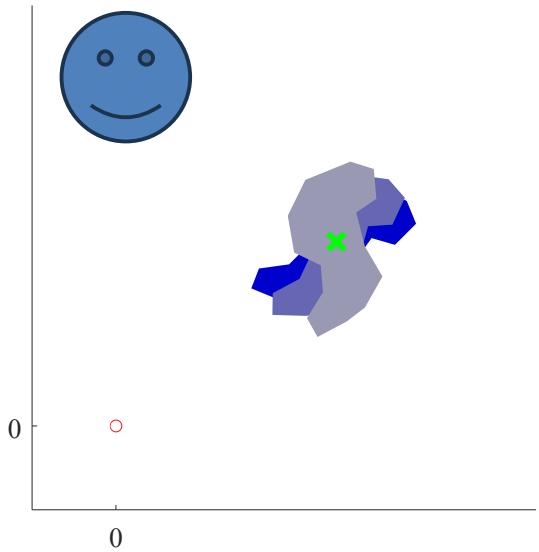
$$\mathbf{c} = \frac{1}{m} \int_{\Omega_0} \mathbf{p}\, \rho(\mathbf{p})\, d\mathbf{p}$$

6

To rotate the object more reasonably, we need to find its centre of mass.

Here we consider Ω_0 the “material space” or reference configuration, i.e. the “input shape” of the object without having been moved yet.

Rigid rotation – centre of mass



- Move c.o.m. to origin
- Rotate around origin
- Move back to c.o.m.

Recall: transformation in homogeneous coordinates:

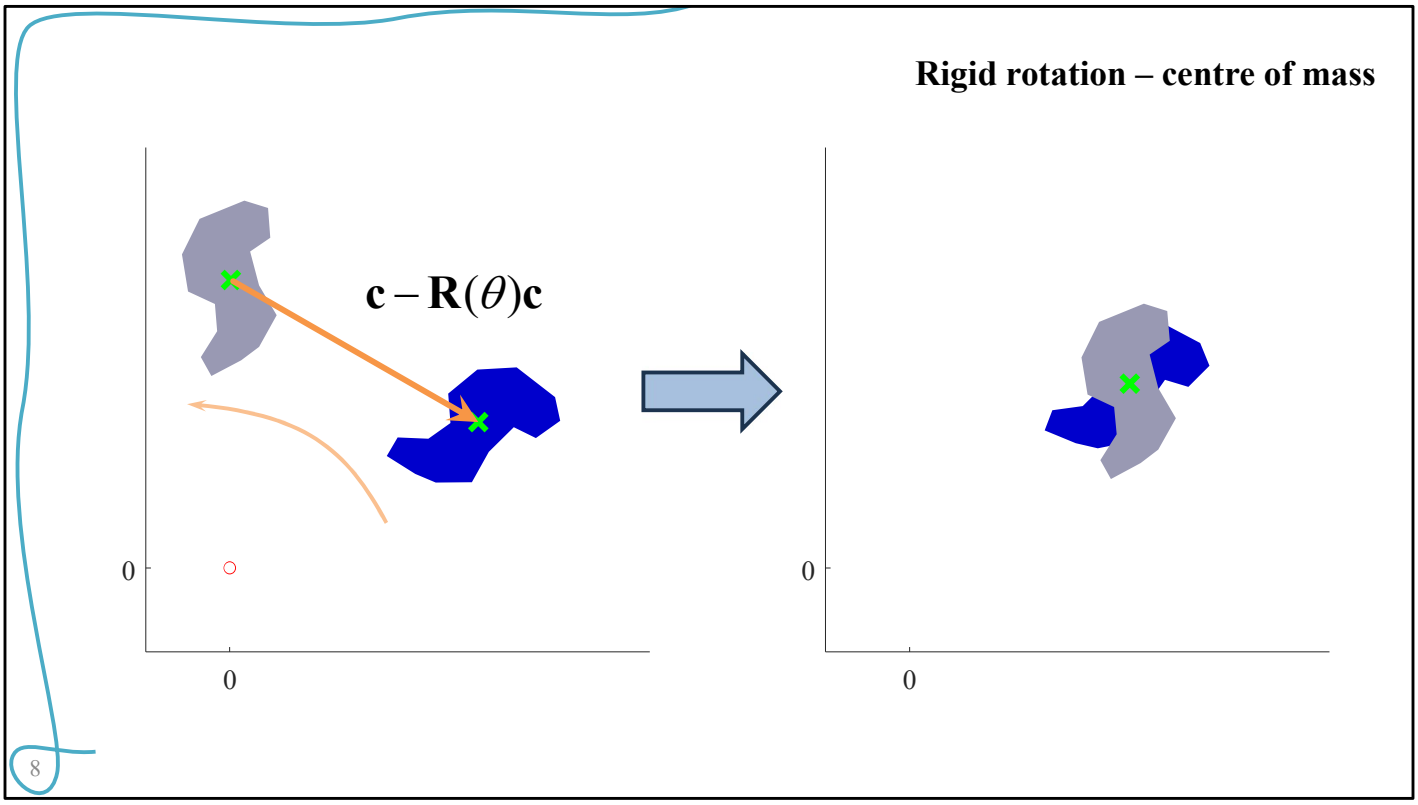
$$\mathbf{R}_{\mathbf{c}}(\theta) = \begin{pmatrix} \mathbf{I} & \mathbf{c} \\ \mathbf{0} & 1 \end{pmatrix} \begin{pmatrix} \mathbf{R}(\theta) & \mathbf{0} \\ \mathbf{0} & 1 \end{pmatrix} \begin{pmatrix} \mathbf{I} & -\mathbf{c} \\ \mathbf{0} & 1 \end{pmatrix}$$

$$= \begin{pmatrix} \mathbf{R}(\theta) & \mathbf{c} - \mathbf{R}(\theta)\mathbf{c} \\ \mathbf{0} & 1 \end{pmatrix}$$

7

Now we can rotate around the centre of mass simply by first moving the object, such that the c.o.m. coincides with the coordinate origin, perform the rotation around the origin as before, and then move the object back, so that the c.o.m. ends up in its original location.

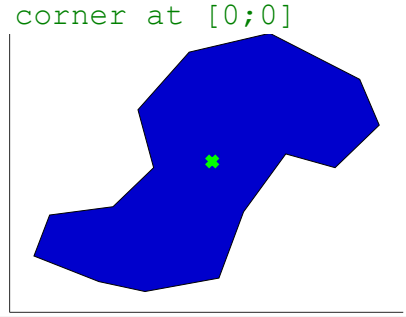
This process can be easily formalized using homogeneous coordinates – which results in a single transformation matrix that describes the rotation around the centre of mass.



Notice that the additional translation by (c-Rc) effectively compensates for the undesired translation that occurs when rotating around the coordinate origin.

Rigid rotation – centre of mass

```
% compute centre of mass (assume unit density)
c = [0;0]; area = 0;
for l=1:size(p,2)
    nl = l+1; if l==size(p,2), nl=1; end

    % connect to origin to form triangle
    tri_area = 0.5 * ( (p(1,l)).*(p(2,nl))-(p(2,l))*(p(1,nl)) );

    % sum weighted centroids and areas (= mass at unit density)
    c = c + tri_area*(p(:,l)+p(:,nl))/3; % third corner at [0;0]
    area = area + tri_area;
end
c = c./area;
```



9

Let's look at how to find the centre of mass in code:

Here's the basic idea for a polygon in 2D … we connect each edge to the origin to form a triangle, compute the area (mass at unit density) and centroid of this triangle, and aggregate.

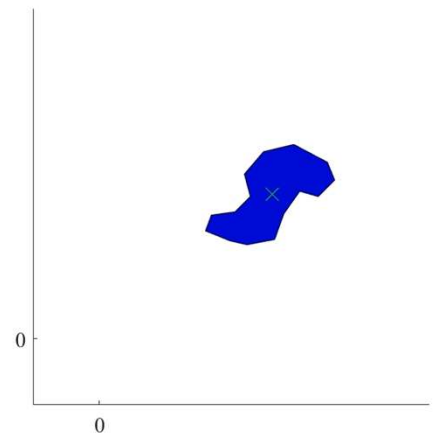
Rigid motion

- No deformations allowed

→ Rotate then translate

- Generalized coordinates (varying over time t):

$$\mathbf{q}(t) = \begin{pmatrix} x(t) \\ y(t) \\ \theta(t) \end{pmatrix} = \begin{pmatrix} \mathbf{t}(t) \\ \theta(t) \end{pmatrix}$$



- Transformation matrix:

$$\mathbf{T}(\mathbf{q},t) = \begin{pmatrix} 1 & 0 & x(t) \\ 0 & 1 & y(t) \\ 0 & 0 & 1 \end{pmatrix} \mathbf{R}_{\mathbf{c}}(\theta(t)) = \begin{pmatrix} \mathbf{R}(\theta(t)) & \mathbf{t}(t) + \mathbf{c} - \mathbf{R}(\theta(t))\mathbf{c} \\ \mathbf{0} & 1 \end{pmatrix}$$

10

Now that we know how to find the centre of mass, and rotate the object around it, we can simply add a translation (after rotation) to describe a rigid motion over time.

Formally, we introduce generalized coordinates that combine translational and rotational degrees of freedom, and allow them to vary over time.

Again, we can write the resulting transformation conveniently in terms of homogeneous coordinates.

We then script a simple example motion for testing → see video.

Rigid body dynamics

- Mapping to world space:

$$\mathbf{p} = \mathbf{R}(\theta(t))\,(\mathbf{p}_0 - \mathbf{c}) + \mathbf{t}(t) + \mathbf{c}$$

- Point velocity:

$$\dot{\mathbf{p}} = \frac{d\mathbf{p}}{dt} = \frac{d\mathbf{R}}{dt}(\mathbf{p}_0 - \mathbf{c}) + \frac{d\mathbf{t}}{dt}$$

- Introduce notation: $\mathbf{v} = d\mathbf{t} / dt,\ \omega = d\theta / dt$ (linear and angular velocity)

- Note that: $\mathbf{R}(\mathbf{p}_0 - \mathbf{c}) = \mathbf{p} - \mathbf{t} - \mathbf{c}$

- We find: $\dot{\mathbf{p}} = \omega \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \mathbf{R}(\mathbf{p}_0 - \mathbf{c}) + \mathbf{v} = [\omega]_\times (\mathbf{p} - \mathbf{t} - \mathbf{c}) + \mathbf{v}$

11

Let's start to formalize physical properties of moving rigid bodies …

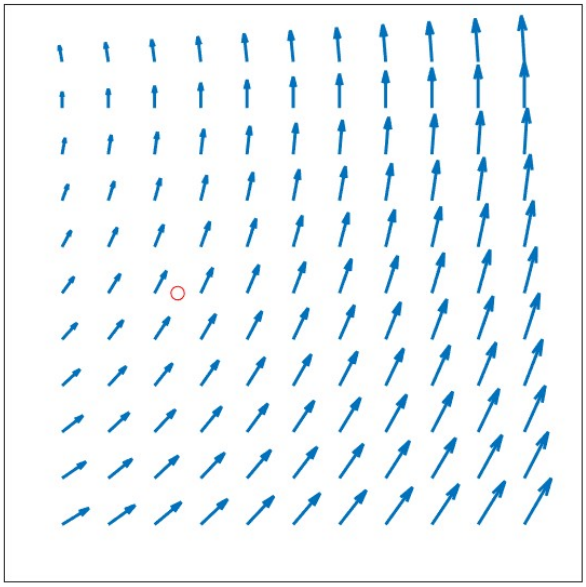
First, we write out the previous transformation that maps any point in material space (p0) to the corresponding position p in world space.

Here, the only time-dependent parameters that define the motion are θ and t (or simply q), whereas p0 and c are material space quantities.

We then take the derivative with respect to time to find the velocity at which the point p moves.

Introducing new variables v and ω for linear and angular velocity respectively, we can simplify the equation as shown.

Velocity field example



$$[\omega]_\times(\mathbf{p}-\mathbf{c}_t)+\mathbf{v}$$

$\mathbf{v} = [0.8;\ 1.6]; \quad \omega = 0.5$

12

Here's an example of what rigid velocity fields look like … note the centre of rotation always remains at the red circle in the images

Notes for German speakers

- Centre of mass → Schwerpunkt (Massenmittelpunkt)
- (Linear) Momentum → Impuls
- Angular momentum → Drehimpuls (Rotationsimpuls)
- Torque → Drehmoment
- Moment of inertia → Trägheitsmoment
- Impulse → Kraftstoß / Impulsübertragung

13

German terminology is confusing, just in case you happen to read a textbook or course notes in German on these topics, be aware of these translations …

Rigid body dynamics

- Linear momentum:

$$
\begin{aligned}
\mathbf{P} &= \int_\Omega \rho \dot{\mathbf{p}}\, d\mathbf{p} \\
&= \int_\Omega \rho([\omega]_\times(\mathbf{p}-\mathbf{t}-\mathbf{c})+\mathbf{v})\, d\mathbf{p} \\
&= \underbrace{\boxed{\int_\Omega \rho[\omega]_\times(\mathbf{p}-(\mathbf{t}+\mathbf{c}))\, d\mathbf{p}}}_{=[\omega]_\times \underbrace{\int_\Omega \rho(\mathbf{p}-\mathbf{c}_t)\, d\mathbf{p}}_{=\int_\Omega \rho\mathbf{p}\, d\mathbf{p} - m\mathbf{c}_t}} + m\mathbf{v}
\end{aligned}
$$

$$
\dot{\mathbf{p}} = \omega\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}\mathbf{R}(\mathbf{p}_0-\mathbf{c})+\mathbf{v} = [\omega]_\times(\mathbf{p}-\mathbf{t}-\mathbf{c})+\mathbf{v}
$$

14

Next, we look at how to compute the total linear momentum of the moving body.

While the term involving v is straightforward, as the integral simplifies to the total mass (notice that v does not depend on the position), the left term is more complicated.

We first move the multiplication by ω out of the integral (also not dependant on position), then look at the remaining integral.

Introducing the notation $c_t = t + c$, we can simplify the right term (again these are not position-dependant quantities).

Rigid body dynamics

- Linear momentum:

$$\mathbf{P} = m\mathbf{v}$$

Rotation around the c.o.m. does not cause linear momentum

- If **c** is the centre of mass in material space, (**t** + **c**) = $\mathbf{c}_t$ is the c.o.m. in world space.
- Consequently, this integral vanishes!

$$\int_\Omega \rho[\omega]_\times(\mathbf{p}-(\mathbf{t}+\mathbf{c}))\,d\mathbf{p} = 0$$

$$\int_\Omega \rho\mathbf{p}\,d\mathbf{p} - m\mathbf{c}_t = 0$$

Definition of c.o.m. applied to world space

15

Notice that, so far, we could have chosen any point c to rotate around and all the calculations on the previous slides would have worked just fine.

Now here is why the centre of mass is a special point:
if c is the centre of mass in material space, and therefore t + c = c_t is the c.o.m. in world space (the current c.o.m. of the object after having moved), then the last remaining integral is also equal to m c_t.
(I.e. the mass-weighted offset from the center of mass is zero by definition.)

Consequently, the entire integral is zero, and the linear momentum is simply P = m v.

Rigid body dynamics

- Angular momentum

$$\mathbf{L} = \int_\Omega (\mathbf{p} - \mathbf{c}_t) \times \rho(\mathbf{p}) \dot{\mathbf{p}} \, d\mathbf{p}$$

$$= \int_\Omega (\mathbf{p} - \mathbf{c}_t) \times \rho(\mathbf{p}) ([\omega]_\times (\mathbf{p} - \mathbf{c}_t) + \mathbf{v}) \, d\mathbf{p}$$

$$= \int_\Omega \rho(\mathbf{p})(\mathbf{p} - \mathbf{c}_t) \times \underbrace{[\omega]_\times}_{} (\mathbf{p} - \mathbf{c}_t) \, d\mathbf{p} + \underbrace{\int_\Omega \rho(\mathbf{p})(\mathbf{p} - \mathbf{c}_t) \times \mathbf{v} \, d\mathbf{p}}_{}$$

90° rotation & scale by angular velocity

$$(\mathbf{p} - \mathbf{c}_t) \times [\omega]_\times (\mathbf{p} - \mathbf{c}_t) = \omega \left\| \mathbf{p} - \mathbf{c}_t \right\|^2$$

$$= \underbrace{\int_\Omega \rho(\mathbf{p})(\mathbf{p} - \mathbf{c}_t) \, d\mathbf{p}}_{=0 \text{ (for rot. around c.o.m.)}} \times \mathbf{v}$$

$$\boxed{\mathbf{L} = \mathbf{I}_\mathbf{c} \, \omega, \quad \mathbf{I}_\mathbf{c} = \int_\Omega \rho(\mathbf{p}) \left\| \mathbf{p} - \mathbf{c}_t \right\|^2 d\mathbf{p}}$$

Moment of inertia

16

Let's now consider the angular momentum, starting from the general definition. [Note: in 2D the cross product results in a scalar value – in 3D it is a 3D vector!]

We again expand the point velocity in terms of the rigid motion (v, ω).

Using the same argument as before (rotation around the centre of mass), we see that the term on the right vanishes.

In the first term, recall that the product by $[\omega]_x$ means that we're rotating the vector $(p - c_t)$ by 90° (in plane) and then scaling its length by ω.

Therefore, we're left with a cross product of two perpendicular vectors, which equals the product of their lengths.

Consequently, we can write the angular momentum (for rotation around the c.o.m.) in terms of the object's moment of inertia.

Rigid body dynamics

Linear momentum

$$\mathbf{P} = m\mathbf{v}$$

Angular momentum

$$\mathbf{L} = \mathbf{I}_c \omega$$

17

In summary, we arrive at the (probably) well-known expressions for linear and angular momentum.

These expressions remain valid in 3D, although the dimensionality of angular momentum changes, as we'll see later.

Moment of inertia for a polygon in 2D

```
Ic = 0;

for l=1:size(p,2)
    nl = l+1; if l==size(p,2), nl=1; end

    % connect edge to centre of mass to form a triangle
    a = p(:,l)-c; b = p(:,nl)-c; % --> triangle (a,b,0)

    % integrate p^2 using bary. coords. on reference triangle
    tri_area = 0.5*(a(1)*b(2)-a(2)*b(1));
    Ic = Ic + ( a'*a + a'*b + b'*b ) * (tri_area*2) / 12;
    % (tri_area*2) = A/A0 with A0=1/2 = reference triangle area

end
```

18

Here's a quick example of how we can compute the moment of inertia (for in-plane rotation around the centre of mass) of a polygon (like the one on the previous slides).

We split the polygon into triangles by connecting each edge to the centre of mass, and moving it such that the c.o.m. is at the coordinate origin.

We then need to integrate p^2 = dot(p, p) over each sub-triangle; using barycentric coordinates, p = a*u + v*b + (1-u-v)*0 = a*u + v*b.

This integral can be transformed onto the reference triangle by moving a –> (1 0) and b –> (0 1); note that the transformation requires us to multiply by the ratio of areas between the source and the reference triangle.

On the reference triangle, the integration then becomes int(dot(p , p) du dv with u=0 –>1 and v=0 –>1-u, which leads to the result shown on the slide.

Rigid body dynamics

- Angular momentum
- What if we're rotating around some point other than the centre of mass?
  - Parallel axes theorem: $\mathbf{I}_d = \mathbf{I}_\mathbf{c} + md^2$
    (for pure rotation!)
- General case: angular momentum relative to some fixed point **o**
  - Spin angular momentum (rotation around c.o.m.)
  - Orbital angular momentum (motion of c.o.m. around **o**)

$$\mathbf{L}_\mathbf{o} = \mathbf{I}_\mathbf{c}\,\omega + (\mathbf{c}_t - \mathbf{o}) \times m\mathbf{v}$$

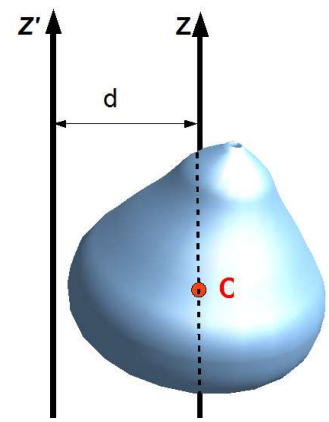


Image: wikimedia (Jack Ver)

19

As a short side note, we can in principle also consider other reference points.

If the object is in pure rotation around some other point, we can use the parallel axes theorem to determine the increased moment of inertia for such a rotation.

For some general motion, we can compute the angular momentum relative to a fixed point **o** by computing the spin and orbital angular momenta separately.

Rigid body dynamics

- Equations of motion in 2D

- Recall generalized coordinates and velocities

$$\mathbf{q} = (x, y, \theta)^{\mathsf{T}}, \quad \dot{\mathbf{q}} = (v_x, v_y, \omega)^{\mathsf{T}}$$

- Linear and angular momentum are conserved (absent external forces)
- Let's add an external force at some point **p**
- Power (force times velocity) must be independent of coordinate system: $\mathbf{f}(\mathbf{p}) \cdot \dot{\mathbf{p}} = \mathbf{f}_{\mathbf{q}} \cdot \dot{\mathbf{q}}$

$$\mathbf{f}(\mathbf{p}) \cdot \mathbf{J}\dot{\mathbf{q}} = (\mathbf{f}(\mathbf{p})^{\mathsf{T}} \mathbf{J})\dot{\mathbf{q}} \qquad \dot{\mathbf{p}} = [\omega]_{\times}(\mathbf{p} - \mathbf{c}_t) + \mathbf{v}$$

$$= (\mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{p}))^{\mathsf{T}} \dot{\mathbf{q}} = \mathbf{f}_{\mathbf{q}} \cdot \dot{\mathbf{q}} \qquad = \mathbf{J}\dot{\mathbf{q}}, \quad \mathbf{J} = \left(\mathbf{I}, \quad [1]_{\times}(\mathbf{p} - \mathbf{c}_t)\right)$$

- Generalized force:

$$\mathbf{f}_{\mathbf{q}} = \mathbf{J}^{\mathsf{T}}\mathbf{f}(\mathbf{p}) \rightarrow \mathbf{f}_{\mathbf{c}} = \mathbf{f}(\mathbf{p}), \quad \tau = ([1]_{\times}(\mathbf{p} - \mathbf{c}_t)) \cdot \mathbf{f}(\mathbf{p}) = (\mathbf{p} - \mathbf{c}_t) \times \mathbf{f}(\mathbf{p})$$

21

We now have all the ingredients we need to describe physical motions of rigid bodies.

We first note that both linear and angular momentum are conserved in the absence of external forces (in 2D this simply means that dq/dt won't change, more on 3D in a moment).

How can we add some external force and compute its effect on the generalized velocities?
We need to find generalized forces, such that the effect of the external force f(p) applied to point p on the rigid body is correctly described in terms of generalized coordinates.

To this end, we observe that the total power injected by this force must be the same regardless of which parameterization we use.
So, we equate the observed power (force times velocity at point p) with the power described by some – yet to be determined – generalized forces (f_q) applied to q.

A few simple arithmetic operations lead to the expression $f_q = J^T f$, with the Jacobian J mapping generalized velocity to local velocity at p.
Finally, we can split these generalized forces into linear forces (equal to f(p)) and torque applied to the centre of mass.

Rigid body dynamics

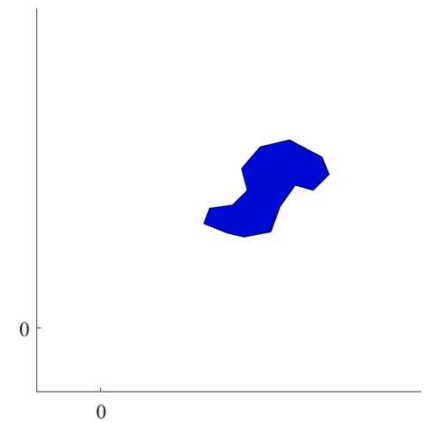
- Equations of motion in 2D

$$\frac{d\mathbf{P}}{dt} = m\dot{\mathbf{v}} = \mathbf{f_c}, \quad \frac{d\mathbf{L}}{dt} = \mathbf{I_c}\dot{\omega} = \tau \qquad \mathbf{M}\ddot{\mathbf{q}} = \mathbf{f_q}, \quad \mathbf{M} = \begin{pmatrix} m\mathbf{I}_{2\times 2} & 0 \\ 0 & \mathbf{I_c} \end{pmatrix}$$

- Basic time stepping (symplectic Euler)

$$\dot{\mathbf{q}}_{t+1} = \dot{\mathbf{q}}_t + \mathbf{M}^{-1}\boxed{\Delta_t \mathbf{f_q}} \quad \text{Impulse}$$

$$\mathbf{q}_{t+1} = \mathbf{q}_t + \Delta_t\, \dot{\mathbf{q}}_{t+1}$$



$$\mathbf{f_q} = \mathbf{J}^{\top}\mathbf{f}(\mathbf{p}) \qquad \mathbf{f_c} = \mathbf{f}(\mathbf{p}), \quad \tau = (\mathbf{p} - \mathbf{c}_t) \times \mathbf{f}(\mathbf{p})$$

22

Now we can write out the usual equations of Newtonian motion (mass times acceleration = force), which relate the change in linear and angular momentum to the applied force and torque respectively.

This equation can be compacted into the well-known notation by introducing the generalized mass matrix M.

Applying a basic forward Euler time discretization let's us simulate the motion of the rigid body, as shown in the video.

(Notice at some frames we add a predefined force on a specific corner of the polygon, and also let it bounce off the "walls" … but more on collisions later.)

Rigid body dynamics – demo code

```
q = [0; 0; 0]; % generalized coordinates (x y phi)
v = [1; 1.5; 3]; % generalized velocities  (vx vy w)

h = 1/30; % time step
for step = 1:120
    p_ = T(q) * [p; ones(1,size(p,2))];
    c_ = T(q) * [c; 1];

    if  ... % eventually ...
        j_ = ... % add an impulse
        tq = (p_(1,k)-c_(1)) * j_(2) - (p_(2,k)-c_(2)) * j_(1);
        v  = v + [j_/m; tq/Ic];
    end

    q = q + h*v;
end
```

23

Here's a (part of a) basic implementation:

First, we initialize the generalized coordinates and velocities and define the time step.

In each step, we then compute the current positions of all corners of our simulated object, and its centre of mass.

At certain times (or collisions with the walls – but more on collisions later) we add an impulse, and update the generalized velocities accordingly.

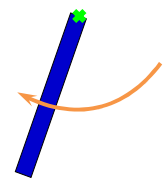
At the end of the time step, we update the generalized coordinates according to the new velocities.

Articulated dynamics

- Can we simulate a whole kinematic chain?

- First step: pendulum → forced to rotate around anchor (instead of c.o.m.)
  - One degree of freedom:

$$\mathbf{T}_1 = \begin{pmatrix} \mathbf{R}(\theta_1) & \mathbf{x}_0 + \hat{\mathbf{c}}_1 - \mathbf{R}(\theta_1)\hat{\mathbf{c}}_1 \\ \mathbf{0} & 1 \end{pmatrix}$$



  - Centre of mass motion:

$$\mathbf{v}_1 = \dot{\theta}_1 \frac{d\mathbf{T}_1}{d\theta_1}\mathbf{c}_1, \quad \omega_1 = \dot{\theta}_1 \quad \rightarrow \quad \begin{pmatrix} \mathbf{v}_1 \\ \omega_1 \end{pmatrix} = \mathbf{J}_1 \dot{\theta}_1$$

  - Project c.o.m. forces as before:

$$\mathbf{f}_\mathbf{c} \cdot \mathbf{v}_1 = \mathbf{f}_\mathbf{q} \cdot \dot{\mathbf{q}} \rightarrow \mathbf{f}_\mathbf{q} = \mathbf{J}_1^\mathsf{T} \mathbf{f}_\mathbf{c}$$

Linear force and torque at c.o.m.

24

Before we move on to 3D (we'll get there eventually ;-) let's think about articulated dynamics, i.e. motion of multiple rigid bodies joined together by kinematic constraints.

In the first step, we'll just look at a simple pendulum, i.e. a bar fixed to a given location at one end swinging around under gravity.

Clearly, the only degree of freedom in this system is the rotation of the bar around the anchor point. We already know how to describe this motion with a homogeneous transformation matrix and a single parameter (which is of course time-dependent).

From there, we can compute the motion (in terms of position, rotation, as well as linear and angular velocity) of the bar's centre of mass.

Combining the expressions for the linear and angular velocities, we can again write this as a product J q, where J is the Jacobian of the mapping from joint velocity to c.o.m. (generalized) velocity.

By the same argument as before, we can use this Jacobian to map the force (and torque) acting on the c.o.m. to the joint torque f_q.

Articulated dynamics

- Can we simulate a whole kinematic chain?

- First step: pendulum → forced to rotate around anchor (instead of c.o.m.)
  - Equation of motion: c.o.m. → d.o.f.

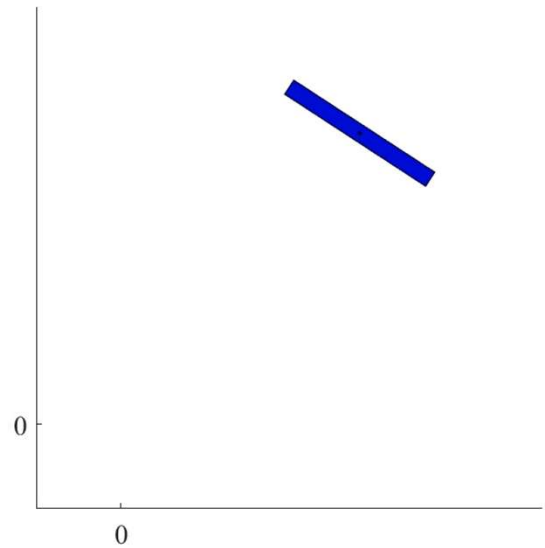
$$\mathbf{M}\ddot{\mathbf{c}}_\omega = \mathbf{f}_\mathbf{c}, \quad \dot{\mathbf{c}}_\omega = (\mathbf{v}_1, \omega_1)^\mathsf{T}$$

$$\ddot{\mathbf{c}}_\omega = \frac{d}{dt}(\mathbf{J}_1\dot{\theta}_1) = \dot{\mathbf{J}}_1\dot{\theta}_1 + \mathbf{J}_1\ddot{\theta}_1$$

$$\mathbf{J}_1^\mathsf{T}\mathbf{M}(\dot{\mathbf{J}}_1\dot{\theta}_1 + \mathbf{J}_1\ddot{\theta}_1) = \mathbf{J}_1^\mathsf{T}\mathbf{f}_\mathbf{c}$$

$$\mathbf{J}_1^\mathsf{T}\mathbf{M}\mathbf{J}_1\,\ddot{\theta}_1 = \mathbf{f}_\mathbf{q} - \mathbf{J}_1^\mathsf{T}\mathbf{M}\dot{\mathbf{J}}_1\,\dot{\theta}_1$$

$$\mathbf{f}_\mathbf{c}\cdot\mathbf{v}_1 = \mathbf{f}_\mathbf{q}\cdot\dot{\mathbf{q}} \rightarrow \mathbf{f}_\mathbf{q} = \mathbf{J}_1^\mathsf{T}\mathbf{f}_\mathbf{c}$$



25

Similarly, we can project the equations of motion describing the c.o.m. mechanics to the degree of freedom (i.e., the joint angle).

Here, we write c'_ω to denote the stacked vector of both linear and angular velocity at the centre of mass.

Note that taking another time derivative of (J θ'), where both factors are time-dependent, required the product rule of differentiation!
(In this simple case, the additional term on the right-hand side is actually always zero, but that will change soon …)

To complete the derivation, we replace d^2c/dt^2 with the resulting expression for the joint acceleration, and project to configuration space by multiplying by J^T from the left.

This results in a compact equation of the familiar form m a = f … with an additional centrifugal force term.

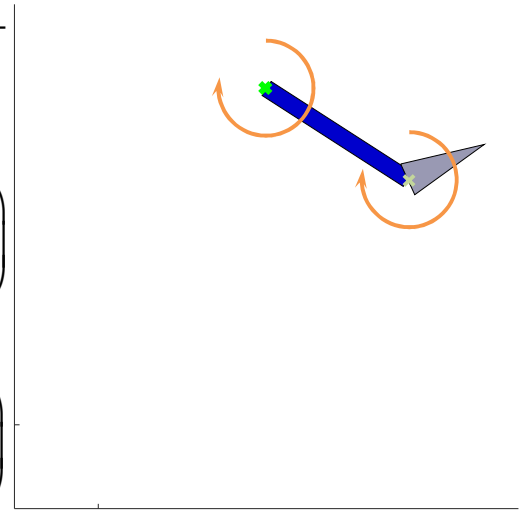
Articulated dynamics

- Can we simulate a whole kinematic chain?
- Next: adding another link
  - Two degrees of freedom: $\mathbf{q} = (\theta_1, \theta_2)^{\mathsf{T}}$
  - Second link transform:

$$\mathbf{T}_2 = \mathbf{T}_1(\theta_1)\begin{pmatrix} \mathbf{R}(\theta_2) & \mathbf{x}_1 + \hat{\mathbf{c}}_2 - \mathbf{R}(\theta_2)\hat{\mathbf{c}}_2 \\ \mathbf{0} & 1 \end{pmatrix}$$

  - C.o.m. motions:

$$\left.\begin{aligned} \dot{\mathbf{c}}_{\omega 1} &= \mathbf{J}_1\dot{\theta}_1 \\ \dot{\mathbf{c}}_{\omega 2} &= \mathbf{J}_{21}\dot{\theta}_1 + \mathbf{J}_2\dot{\theta}_2 \end{aligned}\right\} = \mathbf{J}\dot{\mathbf{q}}, \quad \mathbf{J} = \begin{pmatrix} \mathbf{J}_1 & \mathbf{0} \\ \mathbf{J}_{21} & \mathbf{J}_2 \end{pmatrix}$$



26

Now, let's extend this example to a double pendulum. We add a second link that has basically the same transformation matrix – but this time is chained to the first link (note the multiplication by T_1).

Following the same idea as before, we write out the motions of both centres of mass (for each link) and collect all components into the system's Jacobian.

Note that the motion of the second link also depends on the joint angle at the first link, but not the other way around, resulting in a block-triangular structure in the Jacobian.

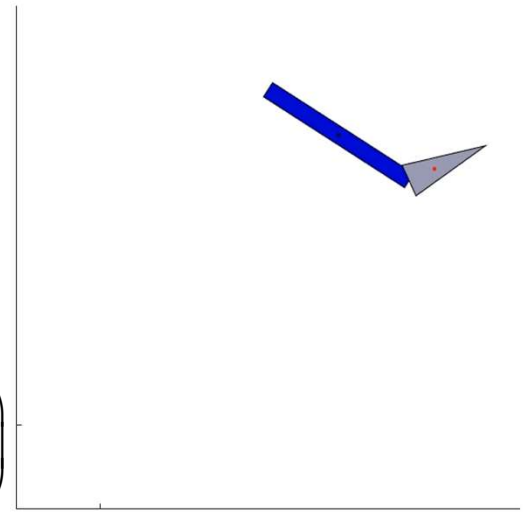
Articulated dynamics

- Can we simulate a whole kinematic chain?
- Next: adding another link
  - Equation of motion as before:

$$\mathbf{J}^{\top}\mathbf{M}\mathbf{J}\ddot{\mathbf{q}} = \mathbf{J}^{\top}\mathbf{f}_{\mathbf{c}} - \mathbf{J}^{\top}\mathbf{M}\dot{\mathbf{J}}\dot{\mathbf{q}}$$

Forces on both centres of mass (stacked)

$$\mathbf{J} = \begin{pmatrix} \mathbf{J}_1 & \mathbf{0} \\ \mathbf{J}_{21} & \mathbf{J}_2 \end{pmatrix}$$



27

The rest of the derivation follows the same steps as the previous example, arriving again at the equation of motion projected into configuration space.

Articulated dynamics – demo code

- System setup:

```
p1  = [0 0 ; 0.2 0 ; 0.2 2 ; 0 2]';
p2  = [0 0 ; 0.4 0 ; 0.2 1 ]’;

[c1,I1,m1] = computeCoMandI(p1, 1.0); % c.o.m and Ic as before
[c2,I2,m2] = computeCoMandI(p2, 1.5);

cr1 = [0.1; 0]; ap1 = [1.9; 4];
R   = @(phi) [cos(phi) -sin(phi); sin(phi) cos(phi)];
T1  = @(q) [R(q(1)+pi) ap1+cr1-R(q(1)+pi)*cr1; 0 0 1];

cr2 = [0.2; 0]; ap2 = [-0.1; 2];
T2  = @(q) T1(q)*[R(q(2)) ap2+cr2-R(q(2))*cr2; 0 0 1];

q   = [1; 1]; % parameters: rotation around anchor points
```


Here’s what the system setup (shapes, transformation matrices, etc.) looks like in Matlab notation.

Articulated dynamics – demo code

- Precompute functions for **J** and d**J**/d*t*:

```
fc = [0; -9.81*m1; 0; 0; -9.81*m2; 0]; % force at c.o.m.s
M = diag([m1 m1 I1 m2 m2 I2]); % mass at c.o.m.s

qs = sym('phi_', [2 1], 'real');
% Jacobian     ... position wrt. parameter      ... rotation
J1 = [[1 0 0 ; 0 1 0] *diff( T1(qs), qs(1)) * [c1; 1] ; 1] ;
J21 =[[1 0 0 ; 0 1 0] *diff( T2(qs), qs(1)) * [c2; 1] ; 1] ;
J2 = [[1 0 0 ; 0 1 0] *diff( T2(qs), qs(2)) * [c2; 1] ; 1] ;
Js = [J1 zeros(3,1) ; J21 J2];
J = matlabFunction( Js, 'Vars',{qs});
% Note: first column in J = effect of q(1) on both c.o.m.s

dq = sym('dq_', [2 1], 'real');
dJs= diff(Js, qs(1))*dq(1) + diff(Js, qs(2))*dq(2);
dJ = matlabFunction(dJs, 'Vars',{qs,dq});
```

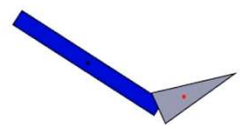
29

Note that Matlab can conveniently mix numerical and symbolic calculations … here we use symbolic variables to automatically compute the required derivatives of the transformations == Jacobian and its time derivative, and convert them into numerical function handles.

Articulated dynamics – demo code

- Time stepping:

```
h = 1/60; % Note: explicit integration can increase energy!
for step = 1:360
    fq = J(q)'*fc ;
    Mq = J(q)'*M*J(q);

    v = v + h*(Mq\(fq - (J(q)'*M*dJ(q,v))*v));
    q = q + h*v;

    % ... update plot and draw
end
```



30

Now we can use these functions to update joint velocities and angles accordingly.

Motions in 3D

- What about 3D?
- Rotations
  - Rotation matrix $\mathbf{R} \in \mathbb{R}^{3\times 3}: \quad \mathbf{R}^{\mathsf{T}}\mathbf{R} = \mathbf{I}, \ \det(\mathbf{R}) = 1$

    Too complicated to parametrize directly
  - Euler angles $\mathbf{R} = \mathbf{R}_z(\psi)\mathbf{R}_x(\theta)\mathbf{R}_z(\phi)$

    Issues with gimbal locking
  - Axis-angle / exponential map

$$\mathbf{R} = \mathbf{I} + \sin(\theta)[\hat{\mathbf{k}}]_\times + (1-\cos(\theta))[\hat{\mathbf{k}}]_\times^2, \quad \|\hat{\mathbf{k}}\| = 1$$

$$\theta = \|\mathbf{k}\|, \quad \hat{\mathbf{k}} = \mathbf{k}/\theta$$

  - Quaternions $\mathbf{r} = (w, x, y, z), \quad \|\mathbf{r}\| = 1$

$$\mathbf{p}_r = \mathrm{im}(\ \mathbf{r}\ (0, \mathbf{p})\ \mathbf{r}^{-1}\ ) \qquad w = \cos(\theta), \quad (x, y, z) = \sin(\theta)\hat{\mathbf{k}}$$

31

In 3D parametrizing rotations is a bit more complicated.
While we generally have three degrees of freedom (local axes to rotate around), a rotation matrix has 3x3 = 9 entries and two additional constraints, which makes it inconvenient to directly work with matrices.

One fairly common approach is to use Euler angles (rotating around three independent axes one after the other, here ZXZ, but other conventions also exist). All of them eventually have issues with gimbal locking, which happens when one of the earlier rotations aligns one axis with another, so we end up with less than three independent axes … now not all rotations away from this state can be described properly.

Another useful approach is to use an axis-angle representation (often a “rotation vector” k, whose length is the angle of rotation, while its direction defines the axis to rotate around). From there we can construct either a rotation matrix using Rodrigues’ formula, or a quaternion – both can be used to apply the rotation to any given vector.

Quaternions are frequently used to describe rotations, they still need to be normalized to describe proper rotations, but otherwise provide a fairly convenient way to apply rotations without expanding to matrix form.
For quaternion rotation: im(…) reads out the imaginary part (x, y, z), and r^-1 = (w, -x, -y, -z)

Motions in 3D

- What about 3D?
- Rotations
  - Rotation matrix — Too complicated to parametrize directly
  - Euler angles — Issues with gimbal locking
  - Axis-angle / exponential map
  - Quaternions — 3D rotation → 3 degrees of freedom; more variables → additional constraints

Motions in 3D

- Revisit for 3D:
  - Mapping to world space
  - Point velocity

$$\mathbf{q} = \begin{pmatrix} \mathbf{t}(t) \\ \mathbf{r}(t) \end{pmatrix}$$

Generalized coordinates

$$\mathbf{p} = \mathbf{R}(\mathbf{q}(t))\,(\mathbf{p}_0 - \mathbf{c}) + \mathbf{t}(\mathbf{q}(t)) + \mathbf{c}$$

Some parametrization of rotation

$$\dot{\mathbf{p}} = [\omega]_\times (\mathbf{p} - \mathbf{c}_t) + \mathbf{v}$$

$$= \omega \times (\mathbf{p} - \mathbf{c}_t) + \mathbf{v}$$

Actually a cross product in 3D

35

Let's revisit how we map a point on a rigid body to its world space position, and what the velocity of that point is when the generalized coordinates change over time …

As in the 2D case, we can still decompose all rigid velocity fields into a linear velocity (motion of the centre of mass) and a rotational component (rotation around the c.o.m.)

In 3D the notation also works out with the actual vector cross product (remember in 2D the cross product would reduce to a scalar, hence the skew-symmetric matrix notation).

Rigid body dynamics in 3D

- Linear momentum $\mathbf{P} = m\mathbf{v}$
- Angular momentum $\mathbf{L} = \mathbf{I}_\mathbf{c}(\mathbf{q})\,\omega$

Angular velocity in 3D (world) space!

State dependent!

- Moment of inertia $\mathbf{I}_\mathbf{c}(\mathbf{q}) = \mathbf{R}(\mathbf{q})\,\mathbf{I}_0\,\mathbf{R}^\mathsf{T}(\mathbf{q})$

(body assumed centred,
i.e. c.o.m. at 0 in body space) $\mathbf{I}_0 = \int_{\Omega_0} \rho(\mathbf{p})[\mathbf{p}]_\times^\mathsf{T}[\mathbf{p}]_\times d\mathbf{p}$

Skew-symmetric "left cross product" matrix

$$[\mathbf{p}]_\times = \begin{pmatrix} 0 & -p_z & p_y \\ p_z & 0 & -p_x \\ -p_y & p_x & 0 \end{pmatrix} \quad \rightarrow \quad [\mathbf{p}]_\times \mathbf{a} = \mathbf{p} \times \mathbf{a}$$

- See also Baraff 1997 SIGGRAPH course notes

36

With the form of the velocity from the previous slide, we can again define the linear and angular momentum analogous to the 2D case.

However, a few additional issues arise:

First, the moment of inertia is now dependent on the current configuration – in 2D there is only one way to rotate, so this was simply a scalar, similar to mass;
in 3D, it is a matrix that needs to be rotated with the object (when computing angular momentum in world space).

To compute the moment of inertia, we first define it in the body's reference space, where it follows a similar structure as in 2D – the square distance from the origin weighted by density – but the math works out to the skew-symmetric form of the point vectors here. (We skip the details here.)

Finally, we also need to find the (world-space) angular velocity …

Rigid body dynamics in 3D

- What is the angular velocity?

Angular velocity in 3D (world) space!

$$\mathbf{L} = \mathbf{I}_c(\mathbf{q})\,\omega$$

$$\dot{\mathbf{r}} \overset{!}{\neq} \omega$$

$$\mathbf{q} = \begin{pmatrix} \mathbf{t} \\ \mathbf{r} \end{pmatrix} \quad \rightarrow \quad \dot{\mathbf{q}} = \frac{d\mathbf{q}}{dt} = \begin{pmatrix} \mathbf{v} \\ \dot{\mathbf{r}} \end{pmatrix}$$

Change of rotation parameters over time

37

In 2D we could compute the angular velocity simply by the time-derivative of the angle by which the object is rotated … 3D rotations, however are a bit more complicated, as we've seen earlier.

So, depending on the chosen parametrization of the 3D rotation, we need to compute the angular velocity that results from changing these parameters over time.

Note that in general the 3D angular velocity is NOT the same as the time derivative of the rotation parameters!

Rigid body dynamics in 3D

- What is the angular velocity?
- Can also compute in material space (i.e., body reference frame)

For quaternions:

$$\omega = \mathrm{im}(\, 2\dot{\mathbf{r}}\mathbf{r}^{-1} \,)$$

$$\omega_B = \mathrm{im}(\, 2\mathbf{r}^{-1}\dot{\mathbf{r}} \,)$$

$$\dot{\mathbf{r}} \overset{!}{\neq} \omega$$

For rotation matrices:

$$[\omega]_\times = \dot{\mathbf{R}}\mathbf{R}^\mathsf{T}$$

$$[\omega_B]_\times = \mathbf{R}^\mathsf{T}\dot{\mathbf{R}}$$

$$\mathbf{q} = \begin{pmatrix} \mathbf{t} \\ \mathbf{r} \end{pmatrix} \quad \rightarrow \quad \dot{\mathbf{q}} = \frac{d\mathbf{q}}{dt} = \begin{pmatrix} \mathbf{v} \\ \dot{\mathbf{r}} \end{pmatrix}$$

38

Without going into too much detail, for the most popular cases, quaternions and rotation matrices (even if the latter further depend on some axis-angle parametrization) the corresponding expressions are relatively straightforward.

For quaternions, recall that im(…) reads out the imaginary (or vector) part of the quaternion while discarding the real part.

For rotation matrices, note that dR/dt may need to be computed based on dr/dt depending on the parametrization.
(Closed-form expressions exist for the exponential map.)

Rigid body dynamics in 3D

- Equations of motion

$$\dot{\mathbf{P}} = \mathbf{f}_c, \quad \dot{\mathbf{L}} = \boldsymbol{\tau}$$

- Euler equation

$$\frac{d}{dt}(\mathbf{I}_c\omega) = \dot{\mathbf{I}}_c\,\omega + \mathbf{I}_c\dot{\omega} = \omega \times (\mathbf{I}_c\omega) + \mathbf{I}_c\dot{\omega}$$

$$m\dot{\mathbf{v}} = \mathbf{f}_c \qquad \mathbf{I}_c\dot{\omega} = \boldsymbol{\tau} - \omega \times (\mathbf{I}_c\omega)$$

- In body reference frame:

$$\mathbf{I}_0\dot{\omega}_B = \mathbf{R}^{\mathsf{T}}\boldsymbol{\tau} - \omega_B \times (\mathbf{I}_0\omega_B)$$

$$(\omega_B = \mathbf{R}^{\mathsf{T}}\omega)$$

39

There's one more step needed to define the rigid body motion in 3D: we need to compute the time derivative of the angular momentum.

Note that both I_c and ω are changing over time (because I_c is rotation dependent). So we need to write out the product rule for the derivative.

After some simplifications (skipping the details here) we arrive at an additional term of the form ω x (I_c ω), which was not present in 2D.

This means that a rotating object exerts a torque on itself when the angular velocity is not exactly aligned with a principal axis (eigenvector) of I_c.

Meanwhile, the equation for linear motion stays the same as in 2D.

We can also move into body reference space by applying R^T from the left, which is sometimes more convenient to implement (because we don't need to update I_0 in each time step).

Rigid body dynamics in 3D

- Equations of motion

$$m\dot{\mathbf{v}} = \mathbf{f}_\mathbf{c} \qquad \mathbf{I}_\mathbf{c}\dot{\omega} = \boldsymbol{\tau} - \omega \times (\mathbf{I}_\mathbf{c}\omega)$$

- External force mapping as in 2D

$$\mathbf{f}_\mathbf{c} = \mathbf{f}(\mathbf{p}), \quad \boldsymbol{\tau} = (\mathbf{p} - \mathbf{c}_t) \times \mathbf{f}(\mathbf{p})$$

- Time step update (depending on selected integration rule):
  - Compute new linear and angular velocity $\mathbf{v}_{i+1}, \omega_{i+1}$
  - Convert to configuration space $(\mathbf{v}_{i+1}, \omega_{i+1}) \rightarrow \dot{\mathbf{q}}_{i+1}$
  - Update state variables $\mathbf{q}_{i+1}$

40

So now we have everything we need to put a basic rigid body simulation together:

First, equations of motion for linear and angular momentum.

Then, we can apply external forces using the same expressions as in 2D when mapping from some point on the body to force and torque on the centre of mass. (In general, the projection into configuration space would still work mostly as in the 2D case for articulated kinematic systems – we'll skip the details here.)

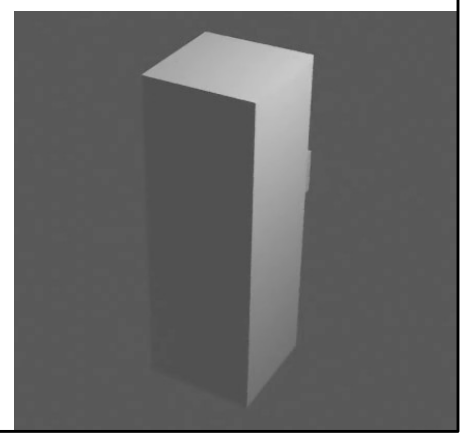
Finally, we can compute a time step, by updating the linear and angular velocities (now including the gyroscopic term in addition to external torque!), then converting angular velocity to rate of change of the rotation parameters, and finally updating the state variables of the rigid body.

Rigid body dynamics in 3D

$$\mathbf{I}_c\dot{\omega} = \boldsymbol{\tau} - \omega \times (\mathbf{I}_c\omega)$$

https://www.youtube.com/watch?v=1VPfZ_XzisU

"The Bizarre Behavior of Rotating Bodies"



41

Note that the **gyroscopic term** also leads to a bunch of unexpected behaviour of rotating objects – such as the 180° flips of this handle (see also "intermediate axis theorem").

Here's a (not so good looking) sim for this effect done in Blender.

Rigid body dynamics - summary

- Conservation of linear and angular momentum
- External forces → centre of mass force and torque
- Integrate linear and angular velocity → update simulation state
- See also: "**A Quick Tutorial on Multibody Dynamics**", C. Karen Liu & Sumit Jain https://fab.cba.mit.edu/classes/865.18/design/optimization/dynamics_1.pdf (02.03.2026)
- Note: instead of forces, we often compute impulses (forces over time) to achieve a specific difference in velocity

→ especially when collisions occur …

42

In summary, rigid body dynamics is governed by conservation of linear and angular momentum.

External forces can be represented as force and torque acting on the centre of mass.

In each time step, we integrate the linear and angular velocity changes over time to find the new velocities and state of the rigid body.

For further details and articulated body motion in 3D, see the notes by Liu and Jain:
https://fab.cba.mit.edu/classes/865.18/design/optimization/dynamics_1.pdf
"A Quick Tutorial on Multibody Dynamics", C. Karen Liu & Sumit Jain, School of Interactive Computing, Georgia Institute of Technology

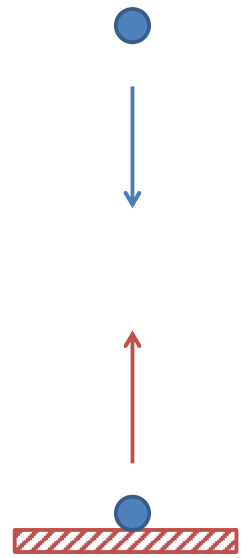
Contact Mechanics

43

Move on to contact mechanics …

Contact constraints

- 1D point mass (x, m) $\quad m\ddot{x} = f$
- Gravity $\quad f_{\text{ext}} = -9.81m$
- Obstacle (floor) $\quad x \geq 0$
- Contact force $\quad f_{\text{floor}} = \lambda$



44

For rigid bodies, we've seen that forces can be moved from points on the body to force and torque on the centre of mass.
To simplify the calculations in this part, we will mostly consider simple point masses (imagine these could be any point on a rigid body that collides with something else).

In the following, we first look at a 1D point mass with position = height x and mass m.

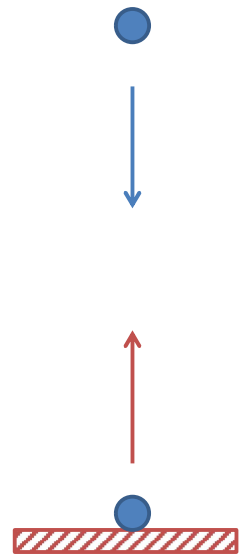
Without any other constraints, this point mass will simply fall down due to gravity.

However, if we introduce a floor at zero height, we have an additional constraint that must always be satisfied.

To ensure a solution exists, the floor may produce a force λ. (Otherwise we cannot solve both equations (m a = f & x >= 0) at the same time.)

Contact constraints

- 1D point mass (x, m) $$m\ddot{x} = -9.81m + \lambda$$
$$x \geq 0$$
- Contact force must never pull $$\lambda \geq 0$$
- No contact force without contact $$x\,\lambda = 0$$



45

Here we first collect the equations from previous slide into a consistent notation.

Note, however, that if λ is unconstrained, any motion could satisfy these equations (we could just make up λ accordingly).
Instead we want this to be a physically reasonable representation of what happens when the object hits the floor.

What do we know about the contact force?

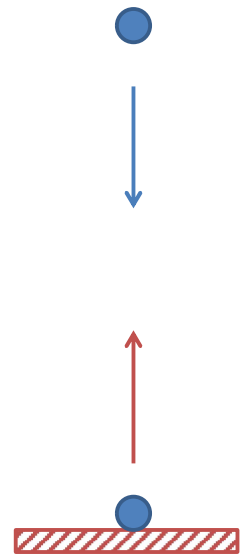
First, it must always push away from the floor, never pull towards it.

Also, if x>0 (no contact) the floor must not produce any force at all.

Contact constraints

- 1D point mass (x, m) $$m\ddot{x} = -9.81m + \lambda$$
$$0 \le \lambda \perp x \ge 0$$

- What happens on impact?
  - Stay on the floor or
  - Bounce back?
  - If so, how fast?



46

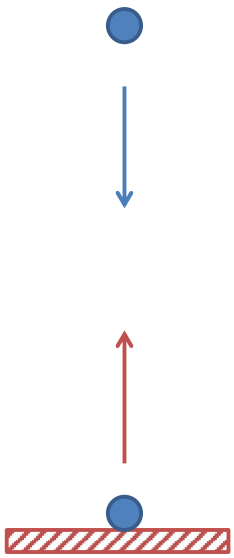
We can write these equations compactly by introducing the shorthand notation for *complementarity* of contact constraint and contact force.

The next question is: What happens on impact?

This question is not really addressed by the model we have constructed so far … (almost) anything could happen.

Contact constraints

- Real world: no perfectly rigid objects
- What happens on impact?
- Rigid model: no deformation allowed



47

In the real world nothing is perfectly rigid, things always deform a little bit when forces act on them.

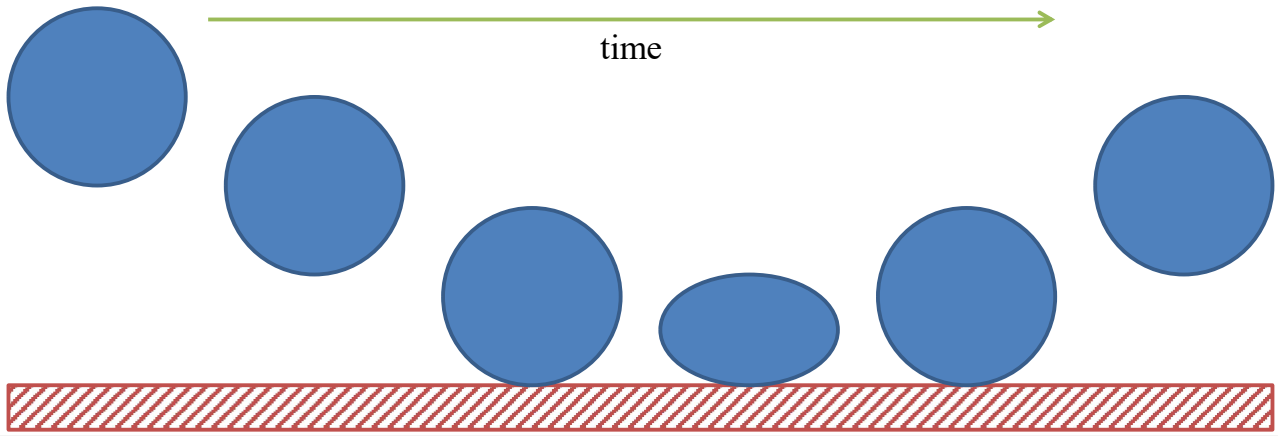
One of the issues with our current model is that nothing as allowed to deform here.

This is why, we can't really deduce from this model, what is supposed to happen on impact.

In order to deal with this problem, let's first look at how contacts happen in an elastic sphere (rather than a rigid point mass) …

Contact constraints

- Real world: no perfectly rigid objects

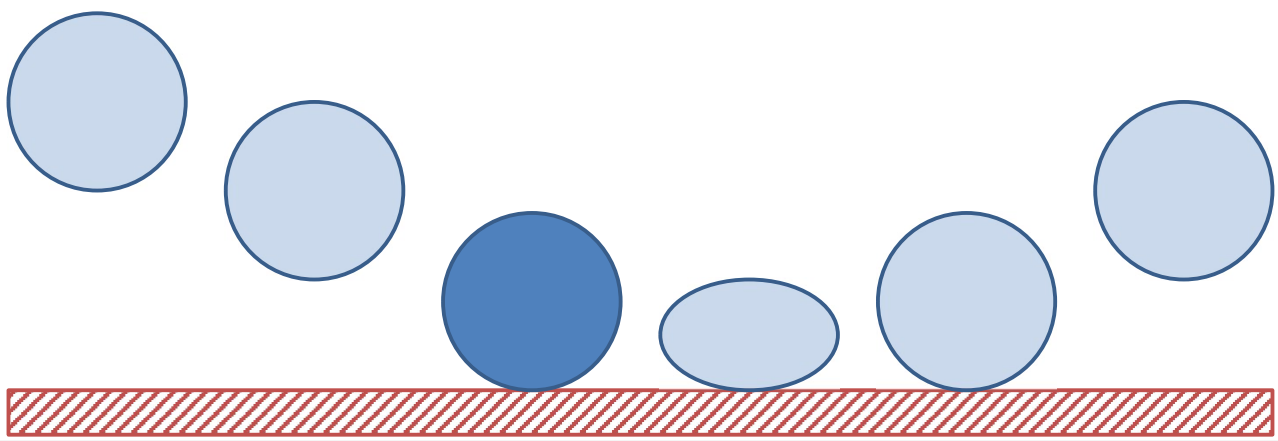


48

If we look at the contact in super slow motion, it probably looks a bit like this …

Contact constraints

- Hertz theory of elastic contact
  - Moving down, surface in contact
  - Contact force causes compressive wave

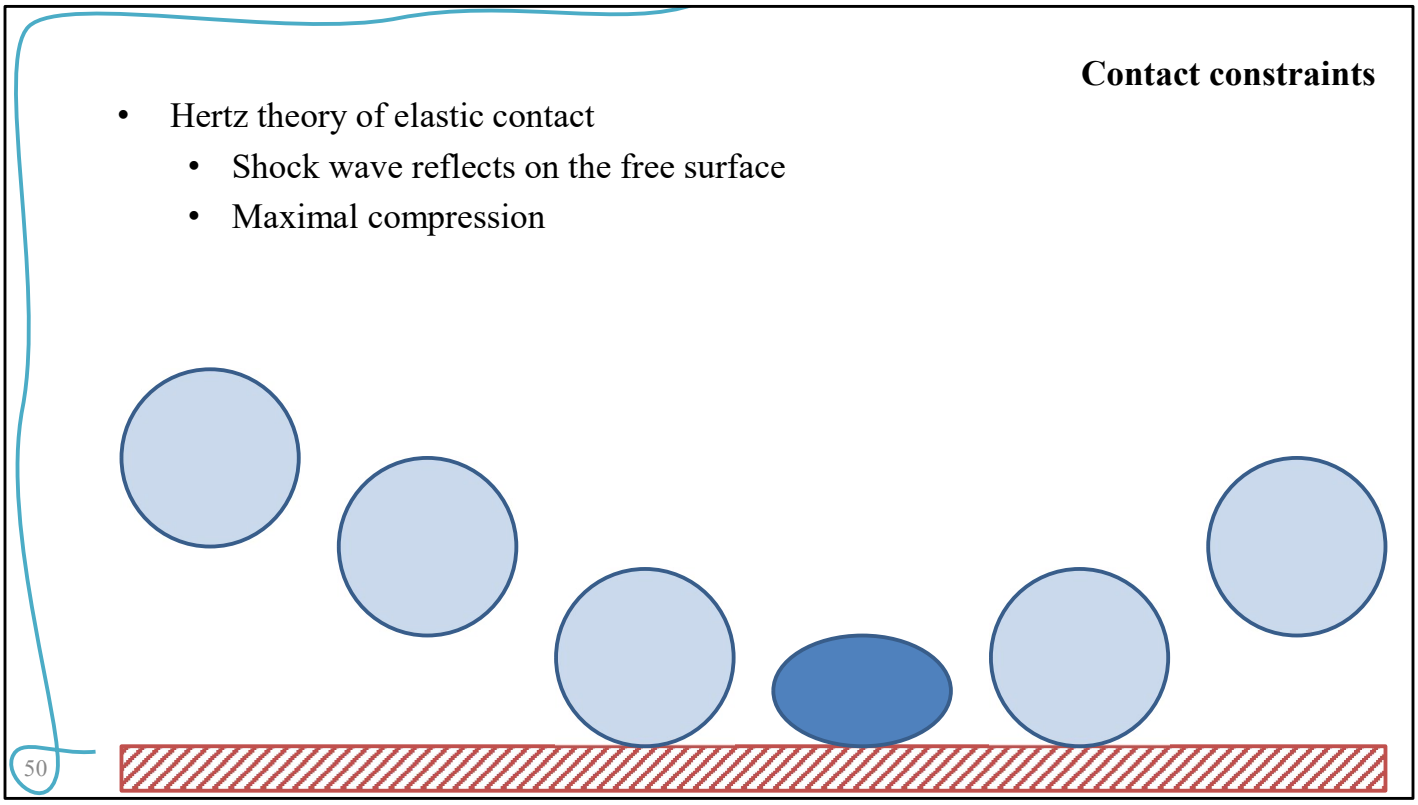


49

Hertz (1881) described contact between an elastic sphere and a rigid plane.

Upon contact, there is a compressive phase where the sphere's kinetic energy is converted to elastic strain energy.

During this phase, a compressive wave travels from the contact point through the sphere.



This shock wave eventually reflects on the free surface and travels back down towards the ground.

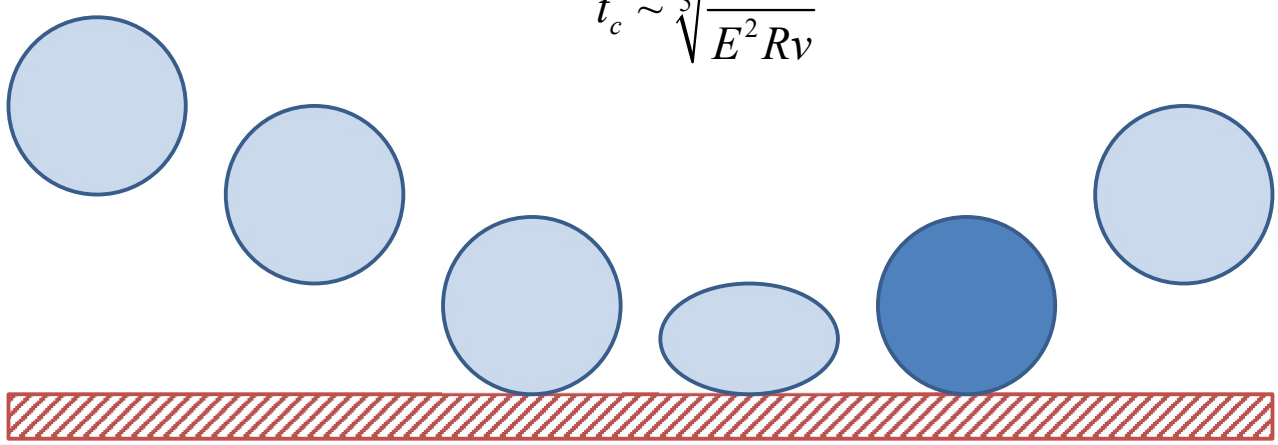
Meanwhile the object reaches a state of maximal compression and minimal kinetic energy.

Afterwards the sphere expands again and pushes away from the ground.

Contact constraints

- Hertz theory of elastic contact
  - Moving back up, contact breaks
  - Duration of contact:

$$t_c \sim \sqrt[5]{\frac{m^2}{E^2 R v}}$$



51

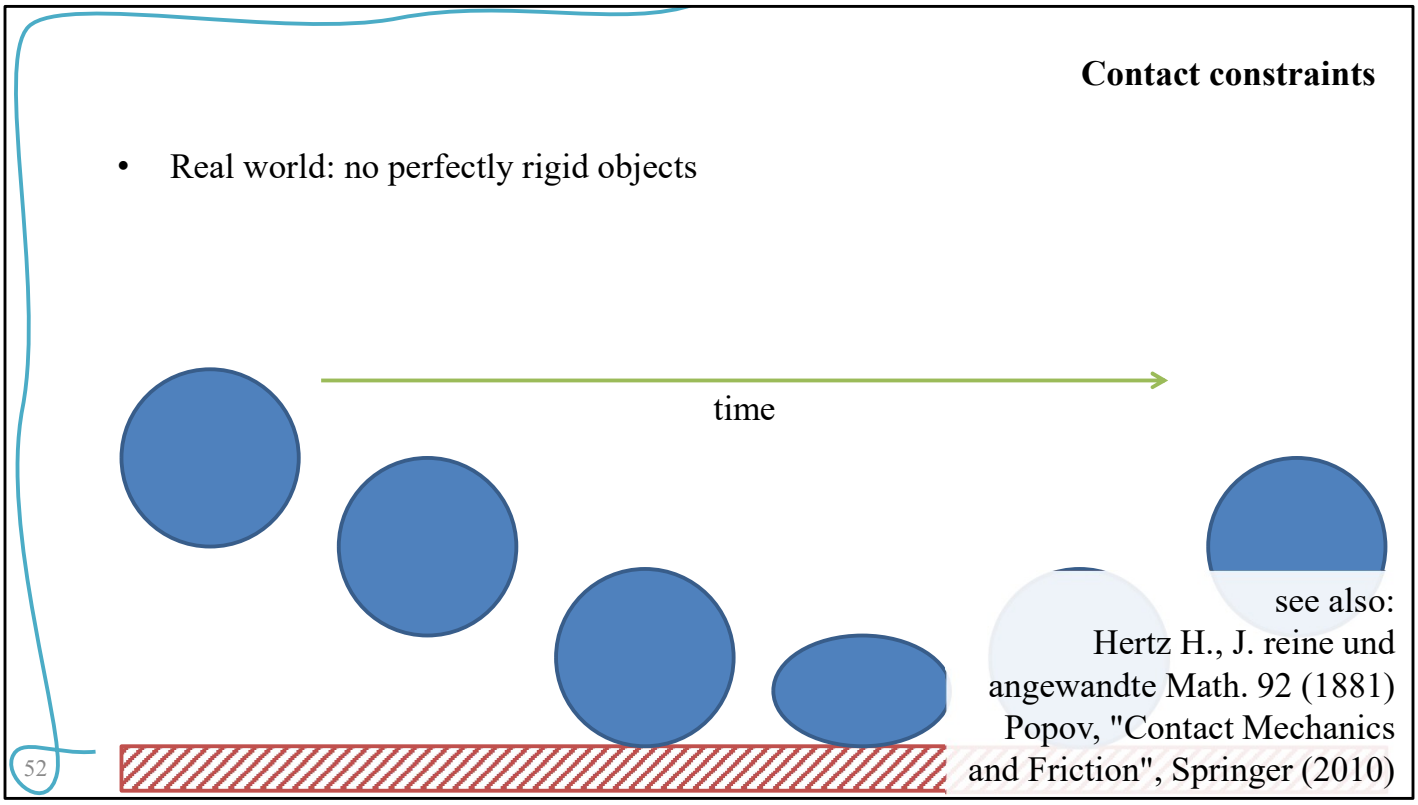
As the sphere starts to move upwards, the contact eventually breaks and the floor no longer exerts a force on the sphere.

Hertz showed how to calculate the duration of this process – from the time of first contact to the time of the contact breaking.

This duration depends on the stiffness (Young's modulus E), size (radius R), mass (m) and velocity of approach (v) of the sphere.

Intuitively, the heavier the sphere is, the longer the contact persists; whereas the stiffer it is, the shorter the contact will last.

In the limit of a rigid sphere (E --> inf) the contact happens (theoretically) instantaneously.



In an ideal model, the total energy of the system is conserved during this process.

This means that the sphere bounces back with equal but opposite velocity of impact.

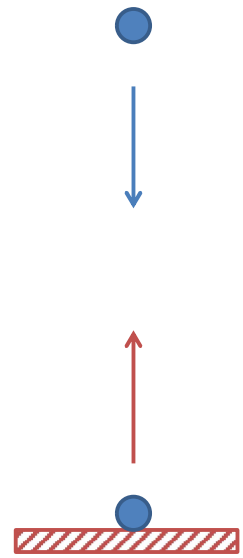
In a real object, some energy is converted into heat due to internal friction and (small) deformations, so the bounce-back is a bit slower.

Contact constraints

- Real world: no perfectly rigid objects

- Rigid model: no deformation allowed

$$m\ddot{x} = -9.81m + \lambda$$
$$0 \le \lambda \perp x \ge 0$$



53

Let's look at the model we have so far again.

The contact force can only be non-zero if x is zero – in a typical time-stepping scheme we would simply stop the point mass at x==0 and it would not bounce back.

So how can we fix our model to be more realistic?

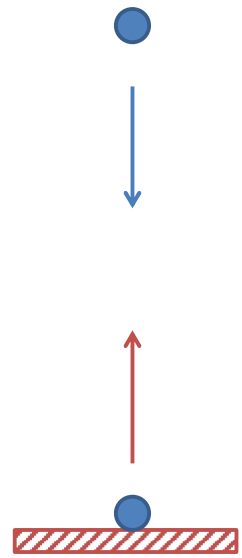
We need to add a new condition to the contact model that describes the instantaneous reflection of velocity due to the contact.

Contact constraints

- Impact law:

$$x = 0 \;\&\; \dot{x}^- < 0:$$

$$\dot{x}^+ = -\dot{x}^-$$



54

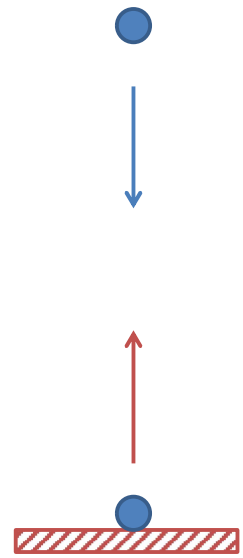
We can formulate this condition as an impact law that applies at the instant of contact.

The velocity just after the contact should be at the reflected velocity just before the contact.

Most systems include a coefficient of restitution here as a simple model for damping (we omit this in the following derivation, but it would be fairly obvious where it should be included).

Contact constraints

- 1D point mass (x, m)



Free flight	In contact
$x > 0:$	$x = 0 \wedge \dot{x}^- < 0:$
$m\ddot{x} = -9.81m$	$m\ddot{x} = -9.81m + \lambda$
	$0 \leq \lambda \perp (\dot{x}^+ + \dot{x}^-) \geq 0$

55

We can include this impact law in the complementarity formulation.

Note that if the contact force is non-zero, the impact law must be fulfilled as an equality constraint.

Time discretization

- BDF1 (aka. implicit Euler)

$$m\ddot{x} = f$$

$$m\dot{v} = f$$

$$\dot{x} = v$$

$$m(v_{i+1} - v_i) = \Delta_t f(x_{i+1})$$

$$x_{i+1} - x_i = \Delta_t v_{i+1}$$

$$\underbrace{mv_{i+1} - mv_i - \Delta_t f(x_i + \Delta_t v_{i+1})}_{r(v_{i+1})} = 0$$

init: $\hat{v} = 0$

$\mathbf{S} = dr / d\hat{v}$

$\Delta_v = -\mathbf{S}^{-1} r(\hat{v})$

$\hat{v} \leftarrow \hat{v} + \Delta_v$

loop while $\|r(\hat{v})\| > \varepsilon$

- How to include constraints?

56

Now we want to simulate this model, so we need a time discretization scheme.

Instead of the basic symplectic Euler scheme we used earlier, here we now introduce the implicit Euler method, also known as BDF1.

This unconstrained version is quite easy to implement. (Use Newton's method to solve if the force is nonlinear.)

But how can we include the contact constraints now?

Time discretization

- BDF1 (aka. implicit Euler)
- How to include constraints?

$$\underbrace{mv_{i+1} - mv_i - \Delta_t f(x_i + \Delta_t v_{i+1})}_{r(v_{i+1})} = 0$$

$$r(v_{i+1}) = \lambda$$

$$0 \le \lambda \perp (x_i + \Delta_t v_{i+1}) \ge 0$$

57

Let's first consider the contact constraints without impact law.

We demand that x (at the end of the time step) is non-negative and add a Lagrange multiplier λ to the right-hand side.

Again λ is non-zero only if x is zero.

If the other forces are constant (such as gravity) or linear, the resulting problem is a linear complementarity problem (LCP).

Time discretization

- BDF1 w/ contact constraints

$$r(v_{i+1}) = \lambda$$
$$0 \le \lambda \perp (x_i + \Delta_t v_{i+1}) \ge 0$$

$$r(\hat{v} + \Delta_v) \approx r(\hat{v}) + \frac{dr}{d\hat{v}} \Delta_v$$

$$\min \left(\frac{1}{2} \Delta_v^\top \frac{dr}{d\hat{v}} \Delta_v + \Delta_v^\top r(\hat{v}) \right)$$
$$\text{s.t. } x_i + \Delta_t (\hat{v} + \Delta_v) \ge 0$$

58

In general, we linearize the residual as in a Newton solver.

The resulting LCP can also be written as a quadratic programming (QP) problem in terms of \Delta_v.

The method is analogous to Newton's method: solve for \Delta_v, update the linearization, and repeat until converged.

(Note: the KKT conditions of the QP result in the (linearized) LCP.)

Time discretization

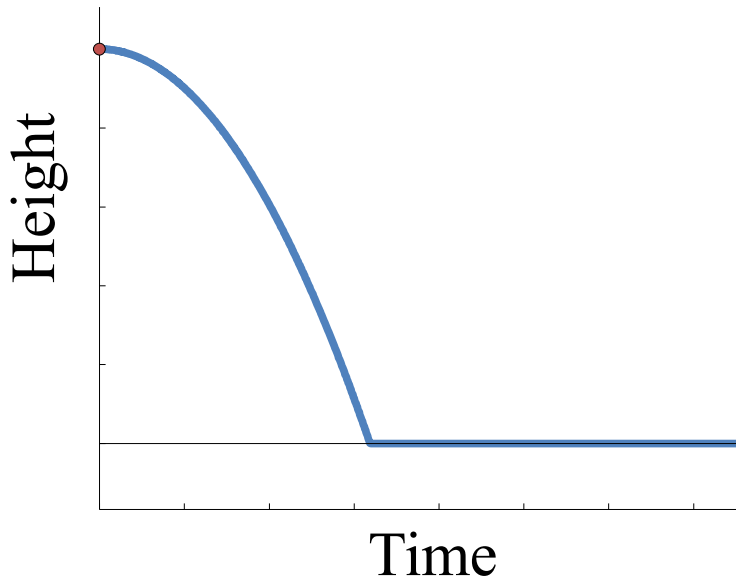
```
vt = 0; % trial velocity
xt = xi + dt*vt; % trial state

for iter=1:maxIters % solver iterations
    r  = (vt-vi)-dt*ag; % residual; vi: before time step, ag: gravity
    dr = 1; % d(r)/d(vt) assuming mass==1
    g0 = gc(xt); dg=dt*dc(xt); % g0: dist. to floor, dg: d(gc)/d(vt)

    dv = quadprog(dr,r,-dg,g0,[],[],[],[],[],oopt); % solve QP

    vt = vt + dv; % update trial velocity
    xt = xi + dt*vt; % update trial state
    if abs(dv)<1e-10, break; end % check convergence
end
```

59

Here is this approach in MatLab code – the complete implementation is in pointMass1D.m (in TUWEL … eventually).
Note: **−**dg because the convention is for less-or-equal constraints

Time discretization

- BDF1 w/ contact constraints



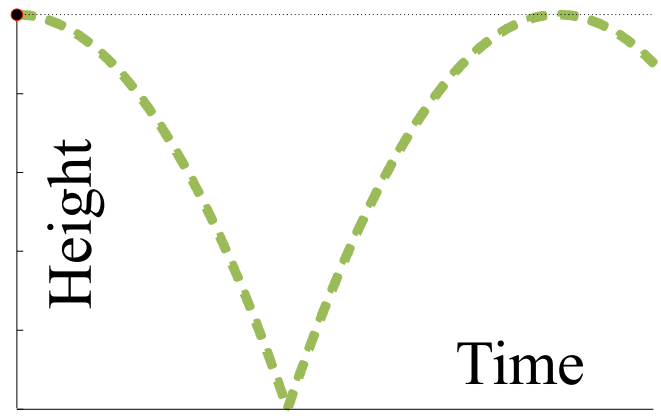
Simulation result: the point mass falls down and gets stuck on the floor …

This is not really a physically reasonable result … we need to implement the impact law correctly!

Time discretization

- BDF1 w/ impact law
  - most time steps unconstrained
  - apply impact law when contact detected

$$x_{i+1} \leq 0 \wedge v_i < 0:$$
$$0 \leq \lambda \perp (v_{i+1} + v_i) \geq 0$$



61

The overall approach remains the same, but we replace the constraint such that we demand reflection of velocity on impact.

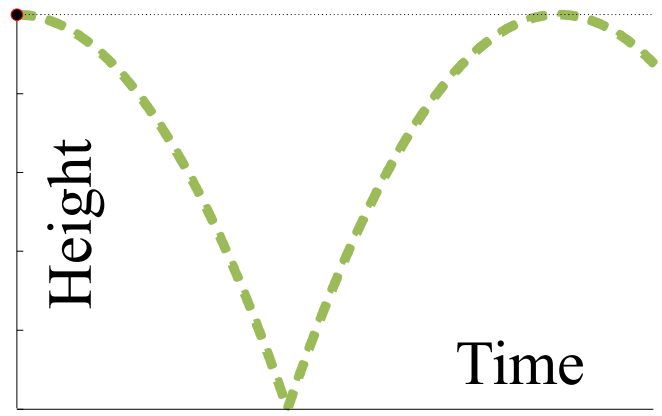
We first solve the unconstrained motion until we detect a contact.

At this time (when x would become negative and velocity is pointing down), we enforce a reflection of the velocity.

Now the simulation result bounces correctly.

Time discretization

```
% ...

if  gc(xt)<0 && vt<0 % impact:   -dv<=2vi <--> (vt+vi)>=0
    [dv,~,~,~,lm] = quadprog(dr,r,-1,2*vi,[],[],[],[],[],oopt);
    lambda = lm.ineqlin; % store contact impulse
else
    dv = dr\(lambda-r);
end


% ...
```



Here's the new constraint in code …

Time discretization

- Soft constraints

$$x \geq 0$$
$$f_{\text{floor}} = \lambda$$

$$x < 0: \quad f_{\text{floor}} = -kx$$
$$x \geq 0: \quad f_{\text{floor}} = 0$$

- Formally unconstrained
- Conservative force (similar to spring)
- Non-linear force!

63

Solving constrained optimization problems can be tedious.

Alternatively, we can replace hard constraints with penalty forces (aka. soft constraints).

This means that we allow a small constraint violation, but add a force that pushes the system towards an admissible state.

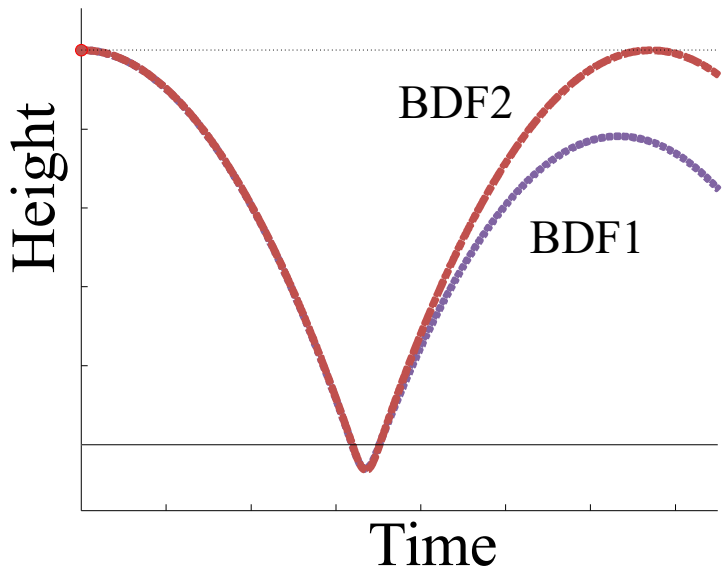
This way, we get a formally unconstrained motion, which we can solve with a standard BDF1 or BDF2 discretization and Newton's method. (Note: for explicit time integration methods, stiff penalties often lead to instability.)

Note that this penalty force allows for some deformation, and is conservative – which means the system should bounce correctly if the solution is accurate.

However, the force is non-linear as there is a kink at x==0.

Time discretization

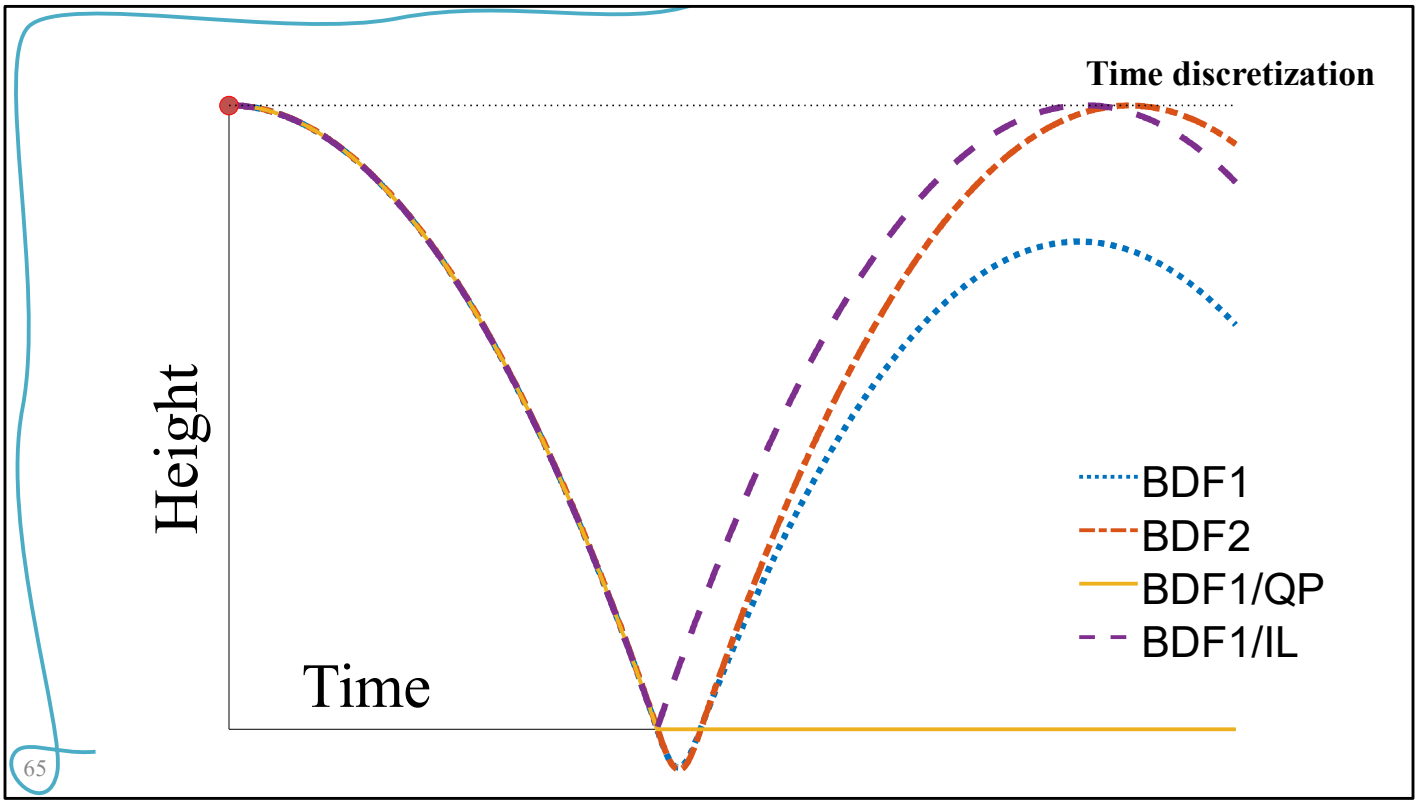
- BDF1, BDF2 w/ soft constraint



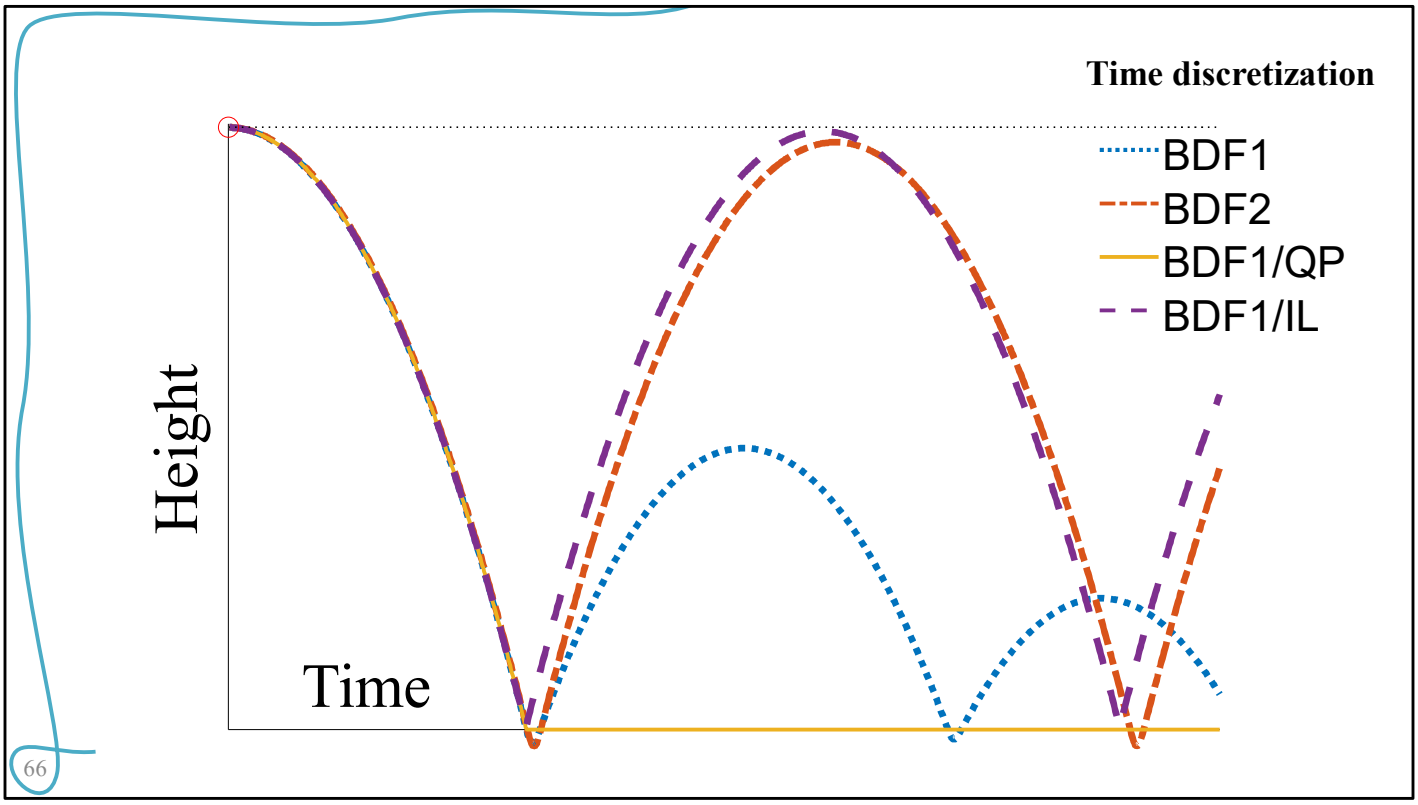
Here are some examples solving this problem with a soft floor and either BDF1 or BDF2 time discretization.

Note that increasing the penalty stiffness (large k) typically leads damping for implicit time integration methods.

On the other hand, a low k leads to noticeable violation of contact conditions.



Here's a comparison of previous results.



And another comparison with an increased penalty factor.

Note that BDF1 leads to strong numerical damping and would need smaller time steps in this example.

In theory, as we increase the penalty stiffness and reduce the time step size, we approach the exact solution.

Summary – Contact constraints

- So far: 1D motion
  Normal to floor

- Rigid contact: complementarity condition
  Restitution modelled via impact law

- Soft contact: allow (small) constraint violation
  Conservative forces – no need for impact law

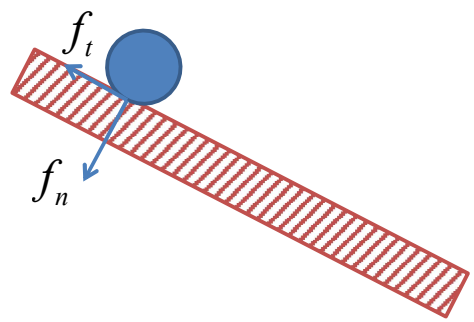
67

Let's recap what we've discussed so far …

Next, we'll look at friction.

Coulomb friction

- Now: tangential motion in contact
- Coulomb model: $f_t \leq \mu f_n$
- Sticking: $f_t < \mu f_n \quad \& \quad v_t = 0$
- Sliding: $f_t = \mu f_n$



68

In the Coulomb friction model, the contact force (normal to the floor) imposes an upper limit on the tangential friction force.

If the total tangential force can be cancelled by an equal and opposite friction force within this limit, the object will not move along the surface.

Otherwise, the object will slide off the surface and encounter the maximal allowed friction force.

The direction of the friction force is always opposite the sliding velocity in this case.

Coulomb friction

- 2D point mass

$$m\ddot{\mathbf{x}} = \mathbf{f}, \quad \mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}, \quad \mathbf{f}_{\text{ext}} = \begin{pmatrix} 0 \\ -9.81m \end{pmatrix}$$

- Floor

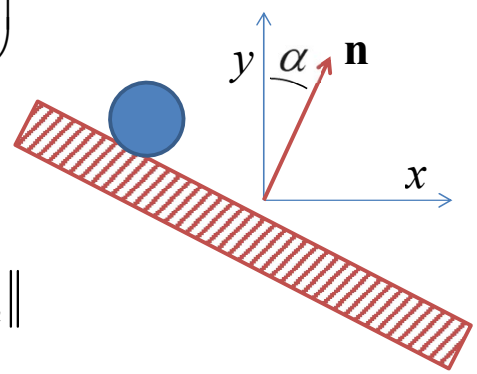
$$\mathbf{n}^{\mathsf{T}}\mathbf{x} \geq 0, \quad \mathbf{n} = \begin{pmatrix} n_x \\ n_y \end{pmatrix} = \begin{pmatrix} \sin\alpha \\ \cos\alpha \end{pmatrix}$$

- Contact force

$$\mathbf{f}_{\text{floor}} = \mathbf{f}_n + \mathbf{f}_t, \quad \mathbf{f}_n = \mathbf{n}\lambda, \quad \|\mathbf{f}_t\| \leq \mu \|\mathbf{f}_n\|$$

- Max. dissipation sliding

$$\max(-\mathbf{f}_t^{\mathsf{T}}\mathbf{T}\dot{\mathbf{x}}) \quad \text{s.t.} \|\mathbf{f}_t\| = \mu \|\mathbf{f}_n\|$$



$$\mathbf{T} = (\mathbf{I} - \mathbf{n}\mathbf{n}^{\mathsf{T}})$$

69

Let's extend our previous model to a point mass in two dimensions.

We'll assume gravity always points along the –y axis.

While the floor can be tilted by some angle α.

The contact force consists of a normal force that enforces the contact constraint, plus a tangential friction force for which we assume Coulomb's model.

Coulomb friction

- Contact force $\mathbf{f}_{\text{floor}} = \mathbf{f}_n + \mathbf{f}_t, \quad \mathbf{f}_n = \mathbf{n}\lambda, \quad \|\mathbf{f}_t\| \le \mu \|\mathbf{f}_n\|$
- 2D: friction force in interval $-\mu\lambda \le f_t \le \mu\lambda$
- 3D: friction force in circle $\|\mathbf{f}_t\| \le \mu\lambda$
  - non-linear, "Coulomb cone"
  - linear approximation: $\mathbf{f}_t = \mathbf{D}\boldsymbol{\beta}, \quad \mathbf{1}^{\mathsf{T}}\boldsymbol{\beta} \le \mu\lambda, \quad \boldsymbol{\beta} \ge 0$

70

In 2D, the friction force can be either to the "left" or to the "right".

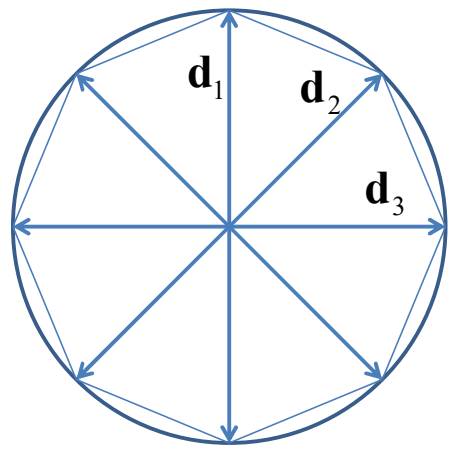
The Coulomb model then imposes a constraint on the tangential force to be in an interval depending on the normal force.

However, in 3D, the friction force can be oriented in any direction along the tangent plane.

This results in a non-linear (aka. conic) constraint; often a linear approximation is used that replaces the cone with a k-sided pyramid.

Coulomb friction

- 3D: friction force
  - linear approximation



$$\mathbf{D} = \left(\mathbf{d}_1, \ldots, \mathbf{d}_m\right)$$

$$\mathbf{f}_t = \mathbf{D}\boldsymbol{\beta}, \quad \mathbf{1}^{\mathsf{T}}\boldsymbol{\beta} \le \mu\lambda, \quad \boldsymbol{\beta} \ge 0$$

71

Image adapted from D. E. Stewart, SIAM Review 2000, 42 (1)

Linear approximation with an 8-sided pyramid …

Vectors d_i are the columns of D.

Coulomb friction

- Simulation time step
  - Collision detection: find contact point and normal
  - Apply impact law
  - Include approximate friction constraints
  - System of equations:

$$\mathbf{r}(\mathbf{v}_{i+1}) = \mathbf{n}\lambda + \mathbf{D}\boldsymbol{\beta}$$ Newtonian motion

$$0 \le \lambda \perp \mathbf{n}^{\mathsf{T}}(\mathbf{v}_{i+1} + \mathbf{v}_i) \ge 0$$ Normal contact

$$\mathbf{1}^{\mathsf{T}}\boldsymbol{\beta} \le \mu\lambda \perp \gamma \ge 0, \quad -\mathbf{D}^{\mathsf{T}}\mathbf{v}_{i+1} \le \gamma\mathbf{1} \perp \boldsymbol{\beta} \ge 0$$ Linearized friction

see also Anitescu & Potra, Nonlinear Dynamics 14 (1997)

72

We now collect all the constraints per contact point …

First, we have the equation of motion,

Then the contact constraint with the impact law,

And finally friction constraints: two orthogonalities, first approx. Coulomb limit, second maximal dissipation (per component of \beta), \mathbf{1} is a vector of ones.

The auxiliary variable \gamma distinguishes static friction (\gamma==0) from dynamic friction.

Coulomb friction

- Simulation time step
  - eliminate free variables
  - combine constraints
  - LCP form

$$\mathbf{A}\mathbf{z} + \mathbf{q} = \mathbf{w}$$

$$0 \le \mathbf{w} \perp \mathbf{z} \ge \mathbf{q}$$

$$\mathbf{z} = \begin{pmatrix} \lambda \\ \boldsymbol{\beta} \\ \gamma \end{pmatrix}, \quad \mathbf{A} = \begin{pmatrix} \mathbf{n}^{\mathsf{T}}\mathbf{S}^{-1}\mathbf{n} & \mathbf{n}^{\mathsf{T}}\mathbf{S}^{-1}\mathbf{D} & 0 \\ \mathbf{D}^{\mathsf{T}}\mathbf{S}^{-1}\mathbf{n} & \mathbf{D}^{\mathsf{T}}\mathbf{S}^{-1}\mathbf{D} & \mathbf{1} \\ \mu & -\mathbf{1}^{\mathsf{T}} & 0 \end{pmatrix}$$

$$\mathbf{r}(\mathbf{v}_{i+1}) = \mathbf{n}\lambda + \mathbf{D}\boldsymbol{\beta}$$

$$\mathbf{r}(\hat{\mathbf{v}} + \Delta_v) \approx \mathbf{r}(\hat{\mathbf{v}}) + \mathbf{S}\Delta_v$$

$$\mathbf{S} := d\mathbf{r} / d\hat{\mathbf{v}}$$

$$\Delta_v = \mathbf{S}^{-1}(\mathbf{n}\lambda + \mathbf{D}\beta - \mathbf{r}(\hat{\mathbf{v}}))$$

see also Anitescu & Potra, Nonlinear Dynamics 14 (1997)

73

We can write the system on the previous slide in a standard LCP form:

First we linearize everything, including the residual, and express \Delta_v explicitly,

Then we replace all unknown velocities with v^\hat + \Delta_v,

And finally substitute \Delta_v with the expression on the top-right;

Carefully expanding all terms results in the matrix A.

The unknown vector z contains the contact and friction forces, as well as the auxiliary variable \gamma.

Coulomb friction

```
% LCP collision resolution
 r = vt-vi-dt*g;
dr = eye(2);

Si = dr\eye(2); % inverse system (mass) matrix
A  = [
    n'*Si*n     n'*Si*D     0
    D'*Si*n     D'*Si*D     ones(2,1)
       cf       -ones(1,2)  0
];
q = [ 2*n'*vi-n'*Si*r  ;  D'*vi-D'*Si*r  ;  0 ];
[w,z] = LCPSolve(A,q);

lambda = z(1); beta = z(2:3); gamma = z(4);
dv = Si*(n*lambda+D*beta-r);
```

74

Here's the same formulation in MatLab code.

Here we're using LCPSolve(…) from:
Andreas Almqvist (2020). A pivoting algorithm solving linear complementarity problems (https://www.mathworks.com/matlabcentral/fileexchange/41485-a-pivoting-algorithm-solving-linear-complementarity-problems), MATLAB Central File Exchange. Retrieved April 3, 2020.

Coulomb friction

- Soft constraints

$$\mathbf{n}^\top \mathbf{x} \geq 0 \qquad \mathbf{f}_n = \mathbf{n}\lambda \quad \Rightarrow \quad \begin{aligned} &\mathbf{n}^\top \mathbf{x} < 0: \quad \mathbf{f}_\mathrm{n} = -k_n\, \mathbf{n}\mathbf{n}^\top \mathbf{x} \\ &\mathbf{n}^\top \mathbf{x} \geq 0: \quad \mathbf{f}_n = 0 \end{aligned}$$

- Sticking

$$\begin{aligned} &\mathbf{T}\dot{\mathbf{x}} = 0 \\ &\mathbf{T} := (\mathbf{I} - \mathbf{n}\mathbf{n}^\top) \end{aligned} \quad \Rightarrow \quad \begin{aligned} &\mathbf{f}_t = -k_t \mathbf{T}\dot{\mathbf{x}} \\ &\text{if } \|\mathbf{f}_t\| < \mu \|\mathbf{f}_n\| \end{aligned}$$

- Sliding

$$\begin{aligned} &\max(-\mathbf{f}_t^\top \dot{\mathbf{x}}) \\ &\text{s.t.} \|\mathbf{f}_t\| = \mu \|\mathbf{f}_n\| \end{aligned} \quad \Rightarrow \quad \mathbf{f}_t = -\mu\, \mathbf{n}^\top \mathbf{f}_n\, (\mathbf{T}\dot{\mathbf{x}} / \|\mathbf{T}\dot{\mathbf{x}}\|)$$

75

Soft constraints are again easier to implement as the system remains formally unconstrained.

We also do not need to approximate the friction cone.

Even though the contact forces have non-smooth points, a Newton solver can efficiently solve this problem (stabilize with line search).

However, soft constraints for sticking can result in noticeable drift over time.

Coulomb friction

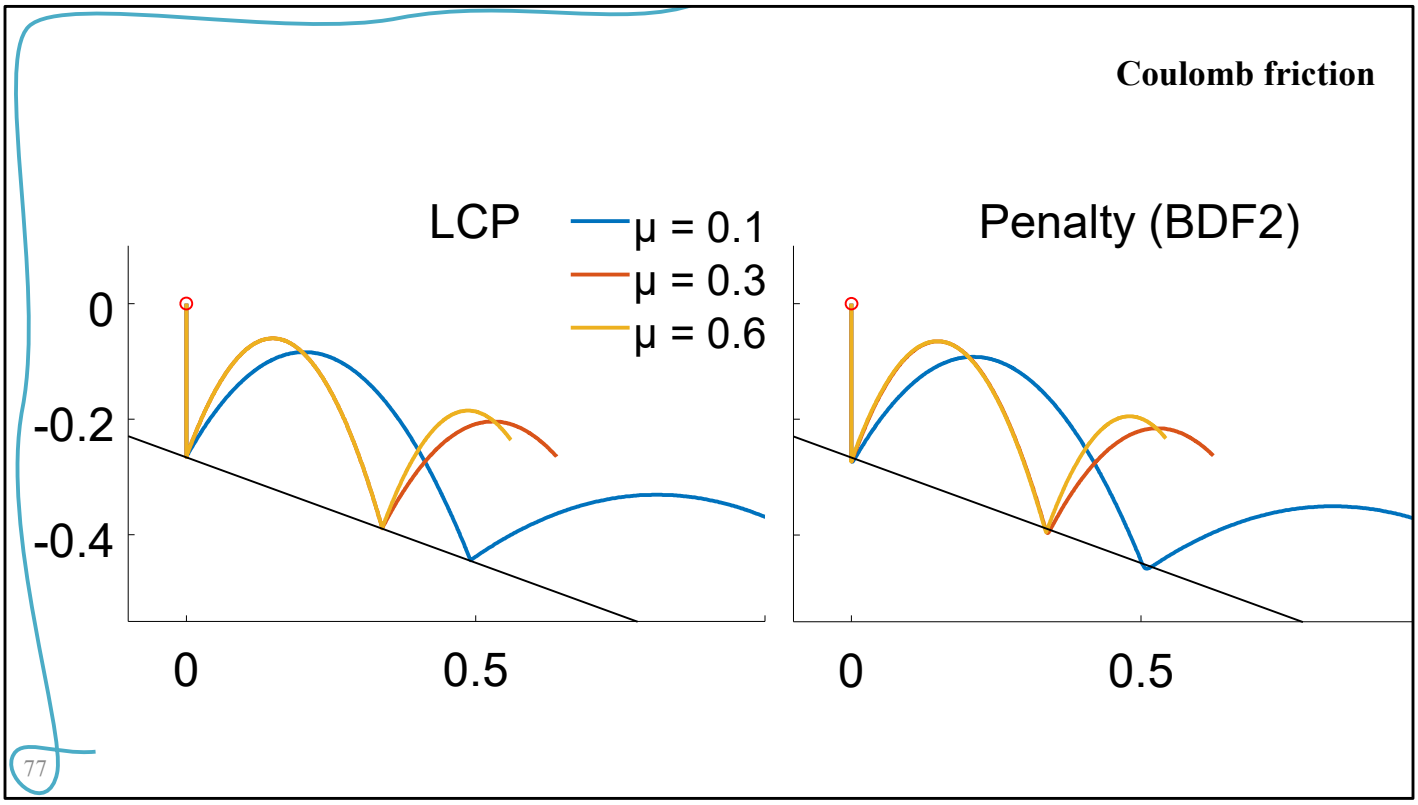
```
% ...

% acceleration with penalty force (mass==1)
a = @(x,v) ([0; -9.81] +fn(x)+ft(v,n'*fn(x)));

% ...

    % solver iteration:
    r = vt-vi-dt*a(xt,vt);
    dr= eye(2)-dt*dt*dadx(xt,vt)-dt*dadv(xt,vt);
    dv= - dr\r;
```

76

Here are parts of the MatLab implementation …

Note that we need to also implement the derivatives of the penalty forces in order to use a Newton solver!

Again, the full implementation will be in TUWEL (eventually) … see pointMass2D.m.



Example results for a 2D point mass with either LCP (left) or soft constraints (right) on a 20° inclined floor;

\mu is the friction coefficient.

Note that when the contact is in the static friction regime, the tangential velocity is zero and the point mass bounces away from the floor along the normal vector (in this example this happens from around μ~0.37 onwards).

Coulomb friction

Extensions

- Separate μ_s for sticking and μ_d for sliding
  → Hysteresis in stick-slip transitions

- Anisotropic models
  See also Erleben et al., "The Matchstick Model for Anisotropic Friction Cones"
  `doi.org/10.1111/cgf.13885`

78

So far we've discussed a very basic friction model … here are some possible extensions.

Summary – Contact and Friction

- So far: point mass examples
- Contact and friction constraints
- Linearization results in LCP
- Practical questions
  - Collision detection
  - “Broad phase” – reduce number of collision tests
  - Contact forces/impulses in rigid body dynamics

[short summary]

Recent work

- Now that we've covered the basics
  … let's look at more recent work in this area

- PBAD https://gamma-web.iacs.umd.edu/PBAD/
  - implicit integration for rigid and articulated dynamics

- IPC https://dl.acm.org/doi/10.1145/3386569.3392425
  - contact detection and intersection-free resolution (mostly penalty based)

- AVBD https://dl.acm.org/doi/10.1145/3731195
  - augmented Lagrange multipliers – halfway between hard and soft constraints

80

Throughout this lecture, we've focused on the basic ideas behind rigid motion and contacts. Of course these topics have been extensively researched in computer graphics and beyond.

To finish up, we'll have a very brief look at some relatively recent papers covering implicit rigid and articulated dynamics, contact handling for deformable models, as well as a GPU-accelerated approach that combines deformable and rigid simulation with a modern contact handling approach in the next few slides.

Recent work

Time Integrating Articulated Body Dynamics Using Position-Based Collocation Methods

Zherong Pan[1] and Dinesh Manocha[2]

https://gamma-web.iacs.umd.edu/PBAD/
https://arxiv.org/abs/1709.04145

Recent work

Time Integrating Articulated Body Dynamics Using Position-Based Collocation Methods

Equation of motion

$$\mathbf{J}^T\mathbf{M}\mathbf{J}\ddot{\mathbf{q}} + \left[\mathbf{J}^T\mathbf{M}\dot{\mathbf{J}} + \mathbf{J}^T\begin{pmatrix} 0 & \\ & [\boldsymbol{\omega}] \end{pmatrix}\mathbf{M}\mathbf{J}\right]\dot{\mathbf{q}} - \mathbf{J}^T\mathbf{f} = 0, \quad (2)$$

where we have $\dot{\mathbf{R}} = [\boldsymbol{\omega}]\mathbf{R}$, $\mathbf{J} = \left(\partial\boldsymbol{\omega}/\partial\mathbf{q}^T \ \partial\mathbf{t}/\partial\mathbf{q}^T\right)^T$, $\mathbf{M}$ being the 6×6 mass

reformulated as energy minimization

$$\mathbf{E}(\mathbf{q}) \triangleq \int_{\mathbf{p}\in\mathcal{B}} \left[\frac{\rho}{2\Delta t^2}\|\mathbf{P}(\mathbf{q}_{k+1}) - 2\mathbf{P}(\mathbf{q}_k) + \mathbf{P}(\mathbf{q}_{k-1})\|^2 + \mathbf{Q}(\mathbf{P}(\mathbf{q}_{k+1}))\right] d\mathbf{p}, \quad (5)$$

where $\mathbf{Q}$ is the potential energy for a position-dependent conservative force $\mathbf{f}$.

82

Recent work

Incremental Potential Contact: Intersection- and Inversion-free, Large-Deformation Dynamics

MINCHEN LI, University of Pennsylvania & Adobe Research
ZACHARY FERGUSON and TESEO SCHNEIDER, New York University
TIMOTHY LANGLOIS, Adobe Research
DENIS ZORIN and DANIELE PANOZZO, New York University
CHENFANFU JIANG, University of Pennsylvania
DANNY M. KAUFMAN, Adobe Research

Incremental Potential Contact: Intersection- and Inversion-free, Large-Deformation Dynamics

Recent work

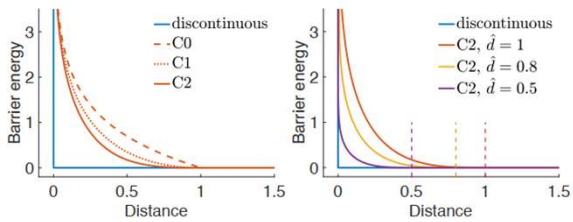
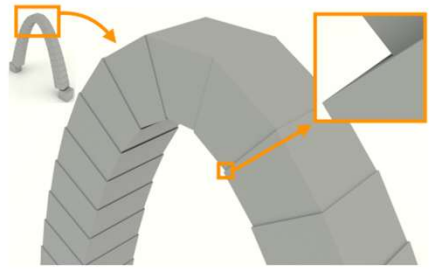


Fig. 3. **Barriers.** Left: log barrier function clamped with varying continuity. We can augment the barrier to make clamping arbitrarily smooth (see our Supplemental). We apply our C^2 variant for best tradeoff: smoother clamping improves approximation of the discontinuous function while higher-order continuity introduces more computational work. Right: our C^2 clamped barrier improves approximation to the discontinuous function as we make our geometric accuracy threshold, $\hat{d}$, smaller.



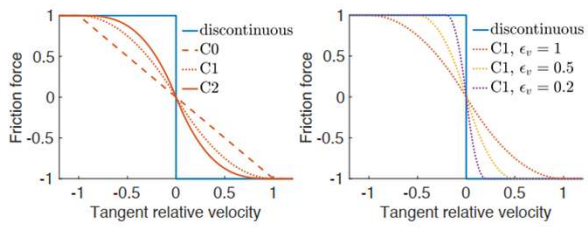


Fig. 6. **Friction smoothing in 1D.** Left: increasing orders of our polynomials better approximate the friction-velocity relation with increasing smoothness. Right: Our C^1 construction improves approximation to the exact relation as we make our frictional accuracy threshold, ϵ_v, and so the size of static friction zone, smaller.

Recent work

Augmented Vertex Block Descent

CHRIS GILES, Roblox, USA
ELIE DIAZ, University of Utah, USA
CEM YUKSEL, University of Utah, USA

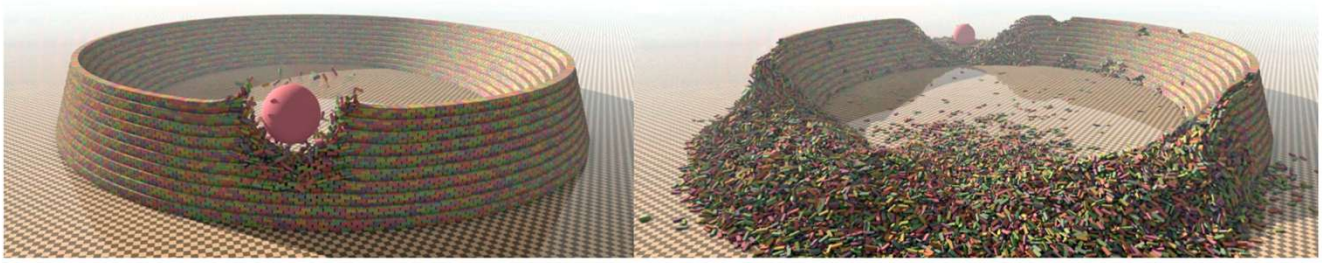


Fig. 1. *Our augmented vertex block descent method allows simulating hard constraints, which are critical for handling contacts and stacking, shown here simulating a pile of 110,000 blocks smashed by a sphere using 4 iterations, taking 3.5 ms (9.8 ms including collision detection) per frame on an NVIDIA RTX 4090 GPU.*

Recent work

Augmented Vertex Block Descent

To support hard constraints with $k_j = \infty$, we use an augmented Lagrangian formulation for the constraint energy, such that for each iteration n we use

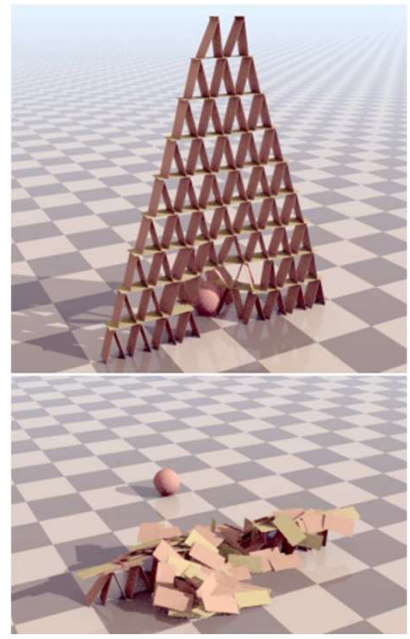
$$E_j^{(n)}(\mathbf{x}) = \frac{1}{2} k_j^{(n)} \left(C_j(\mathbf{x}) \right)^2 + \lambda_j^{(n)} C_j(\mathbf{x}) \,, \tag{8}$$

where $k_j^{(n)}$ is the finite stiffness and $\lambda_j^{(n)}$ the Lagrange multiplier (also called *dual variable*) used for AVBD iteration n. The resulting constraint force is then

$$\mathbf{f}_{ij}^{(n)} = -\left(k_j^{(n)} C_j(\mathbf{x}) + \lambda_j^{(n)} \right) \frac{\partial C_j(\mathbf{x})}{\partial \mathbf{x}_i} \,. \tag{9}$$

the dual variable is updated using

$$\lambda_j^{(n+1)} = k_j^{(n)} C_j(\mathbf{x}) + \lambda_j^{(n)} \,.$$



86